Moderated Learning Management System

CS4433 Software Engineering - Final Project Report

National University of Lesotho

Department of Mathematics and Computer Science

Student: [Full name]

Student number: [Student number]

Supervisor: [Supervisor name]

Repository: [GitHub repository URL] (tags `v1.0`, `v1.1`)

Date: November 2026

Abstract

This report describes how a moderated Learning Management System (LMS) for a secondary school was taken through the full software engineering life cycle using the Waterfall model with V-Model verification and validation. The school needs students to submit assignments, lecturers to mark them, and Heads of Department to moderate every mark before students can see it, with late penalties, individual extensions, resubmission and a complete record of every mark change. Seven conflicts and twelve ambiguities in the stakeholder statements were resolved into a specification of 21 functional requirements, 12 measurable non-functional requirements and 12 business rules. The system was designed as a strict layered architecture using Strategy, Repository, Factory Method, Observer and Template Method patterns, and implemented in C++17 with SQLite and Argon2id password hashing. Verification used 264 automated unit, integration and system tests derived by equivalence partitioning and boundary-value analysis, performance tests with 800 students, and scripted acceptance tests. Line coverage of the business-logic layer was 95.9 % and eight defects were found and closed. After release v1.0 a change request for class-wide extensions was analysed as perfective maintenance, implemented as release v1.1 and regression-tested (278 tests). The report closes with the professional, legal and ethical issues raised by processing learners' data and a critical reflection on the process.

Table of contents

CS4433 Software Engineering - Final Project Report
Abstract
2. Introduction
  2.1 Context and problem
  2.2 Aims
  2.3 Approach
  2.4 Structure of the report
3. Project Management Plan
  3.1 Feasibility
  3.2 Work breakdown and schedule
  3.3 Monitoring and control
  3.4 Risk management
4. Requirements Engineering
  4.1 Stakeholders
  4.2 Conflicts and their resolution
  4.3 Software Requirements Specification
  4.4 Use case model
5. Software Design
  5.1 Architecture
  5.2 Class design, patterns and SOLID
  5.3 Behavioural models
  5.4 Database design
6. Implementation
  6.1 Structure
  6.2 Key decisions
  6.3 Security
  6.4 Persistence
7. Verification and Validation
  7.1 Strategy
  7.2 Test design
  7.3 Results
  7.4 Defects
8. Configuration Management and Quality Assurance
  8.1 Configuration management
  8.2 Quality assurance
9. Deployment
10. Maintenance: Change Request CR1
11. Professional, Legal and Ethical Issues
12. Conclusion
13. References
14. Appendices
  Appendix A - Requirements traceability matrix
  Appendix B - Test results: test report v1.0, defect log, CR1 regression report
  Appendix C - User manuals
  Appendix D - Updated risk register
  Appendix E - Fully dressed use cases UC-02 and UC-03

2. Introduction

2.1 Context and problem

Assessment in the school is still largely paper-based. Lecturers collect work, calculate late penalties by hand and pass marks to Heads of Department (HoDs), who check them before they are published. The process is slow, penalties are applied inconsistently, and when a mark is questioned there is no reliable record of who changed it, when or why. The school asked for a Learning Management System that manages courses, assignments and assessment for four types of user (students, lecturers, HoDs and an administrator), where every user can only perform the actions of their role, every submission follows a defined life cycle, and every change to a mark is recorded.

The constraints are demanding for their simplicity. The system must run offline on the school's old computers, must be usable by teachers who "are not IT experts", must protect passwords, and must respond quickly with all 800 students stored. Web and mobile interfaces, online payment, plagiarism detection and automated grading are out of scope. The system is a C++ console application with a SQLite database.

2.2 Aims

The project aimed to (1) plan and manage a realistic software project; (2) elicit, reconcile and specify verifiable requirements from conflicting stakeholders; (3) produce a justified design with UML, design patterns and a normalised database; (4) implement a maintainable and secure object-oriented application; (5) verify and validate it systematically; (6) apply configuration management and quality assurance; and (7) manage a change after release while considering the professional, legal and ethical implications.

2.3 Approach

The brief prescribes the **Waterfall model** (Royce, 1970) with three phase gates, and the **V-Model** (Forsberg and Mooz, 1991) for verification: each development phase on the left of the "V" has a matching test level on the right that verifies it. Waterfall suits this project because the requirements are relatively stable once the stakeholder conflicts are resolved, the product is small, and the brief requires documented deliverables at each phase. Its weakness, late discovery of misunderstood requirements, was reduced by resolving conflicts explicitly at Gate 1 and by deriving system tests directly from the use cases. Every artefact described here is in the Git repository, and a traceability matrix (Appendix A) links each requirement to its use case, design elements, source files and tests.

2.4 Structure of the report

Section 3 presents the project management plan. Section 4 covers requirements engineering, section 5 the design, section 6 the implementation and section 7 verification and validation. Section 8 describes configuration management and quality assurance, section 9 deployment, section 10 the maintenance change request and section 11 professional issues. Section 12 concludes with a critical reflection.

3. Project Management Plan

3.1 Feasibility

A feasibility study was carried out before committing to the project, covering five dimensions (full study: `docs/03-project-management/project-management-plan.md`).

Technical feasibility was high. SQLite gives transactional, single-file storage with no server, which fits the offline constraint, and the C++ standard library covers everything else except password hashing, for which a small reference library exists. A console application with an embedded database needs very little memory; measured login time on the development PC was between 56 and 153 ms. The main technical risk identified was the toolchain itself, and it did occur (section 3.4).

Operational feasibility depends on non-expert users. Numbered text menus, one task per option, role-specific manuals and a demonstration database for training address this. A single shared computer could become a queue near deadlines; this is mitigated by the speed of submission and recorded as a limitation.

Economic feasibility is favourable. All tools and libraries are free and open source, existing hardware is reused, and the only cash cost is about LSL 300 for backup media. Developer time and training are in-kind. Hosted commercial LMS products would add subscription costs and would fail the offline requirement.

Legal feasibility was assessed against the Lesotho Data Protection Act 2011, which governs the processing of learners' personal information. The design minimises the data held, keeps it on the school's own computer and restricts access by role (section 11). Third-party licences (public domain, CC0/Apache-2.0, BSD-3) allow the intended use.

Schedule feasibility was tight: 36 days of effort in 35 working days, possible only because some tasks run in parallel off the critical path. A MoSCoW scope rule (drop "Could" requirements first) was adopted as the contingency.

The recommendation was to proceed with a single-computer, offline architecture, set up the toolchain and continuous integration early, and treat notifications (a Could requirement) as optional scope.

3.2 Work breakdown and schedule

The Work Breakdown Structure (Figure 1) decomposes the project into nine work packages that mirror the phases of the brief, each ending in a deliverable. The Gantt chart (Figure 2) schedules them over seven weeks from 21 September 2026, with finish-to-start dependencies, three phase gates and four milestones.

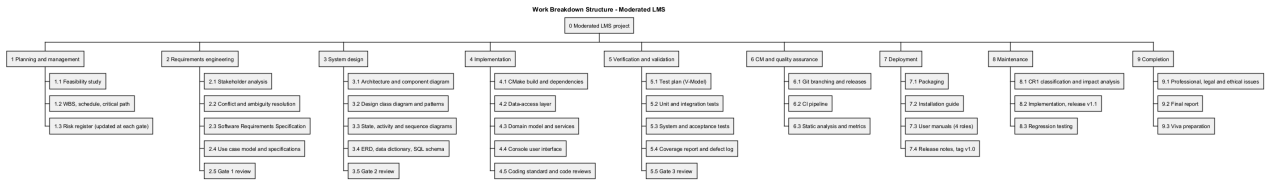


Figure 1 - Work Breakdown Structure

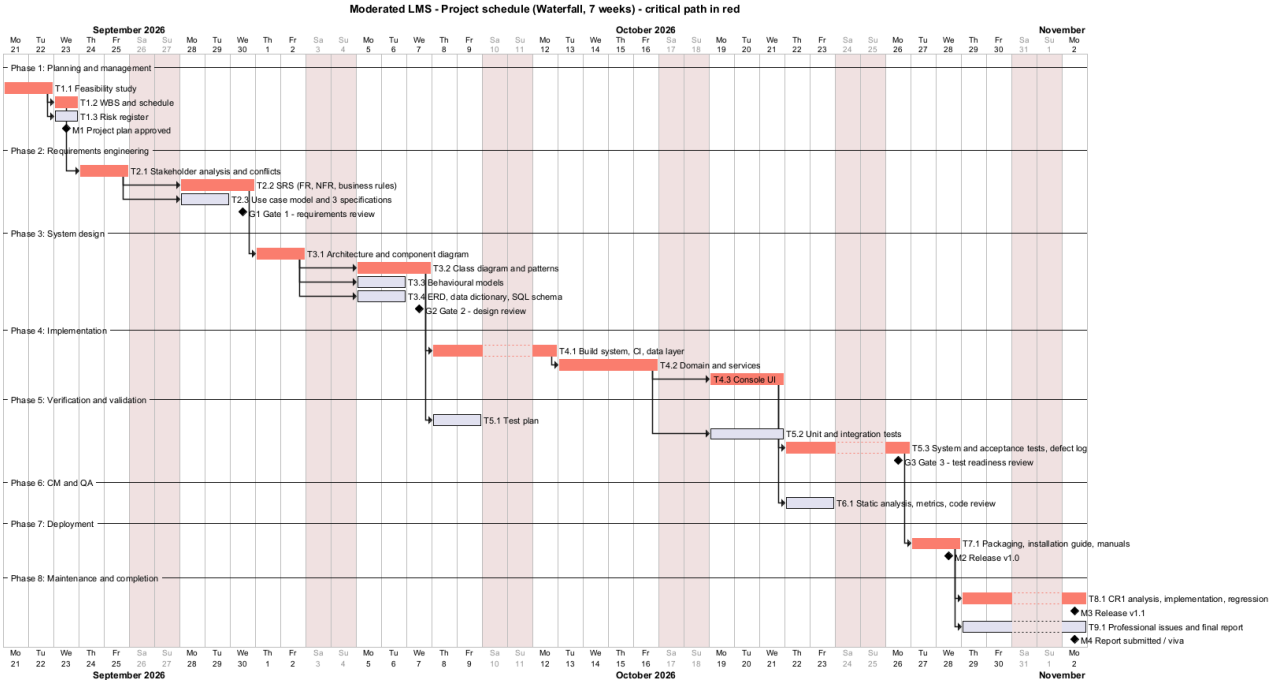


Figure 2 - Gantt chart; critical-path tasks in red

The **critical path** runs through feasibility, WBS, stakeholder analysis, SRS, architecture, class design, the three implementation packages, system testing, deployment and the change request. Tasks off the critical path (the risk register, use-case specifications, behavioural models, database design, test planning, unit testing, quality assurance and report writing) have float and were the planned buffer. The critical path runs through implementation of the business logic before the user interface, so the most uncertain code was written first. It also runs through system testing rather than unit testing: unit tests are written alongside the code and do not by themselves release anything.

3.3 Monitoring and control

Progress was checked weekly against the Gantt chart. Each gate had written exit criteria: at Gate 1, a signed-off SRS with at least 12 functional and 6 measurable non-functional requirements; at Gate 2, a design consistent with the SRS, with no implementation beforehand; at Gate 3, all Must requirements tested, coverage of at least 70 % and no open major defects. Quality gates were automated where possible: the CI pipeline fails on compiler warnings on Linux, on any failing test, on coverage below 70 % and on any cppcheck finding.

A candid note on plan versus actual: the gate dates are the plan. The implementation, tests and documents were in fact produced in a concentrated effort, and the Git history is the authoritative record of when each artefact was created. The documents do not claim otherwise.

3.4 Risk management

The risk register (Appendix D) started with eleven risks across technical, project and people categories. Each has likelihood and impact on a 1-5 scale, exposure (L x I), a mitigation, a contingency and an owner. It was re-scored at every gate. The highest initial exposures were schedule overrun (R2, 20) and errors in deadline and penalty logic (R5, 15), the latter because an error would directly produce unfair marks.

The register earned its place during the project. **R1**, an unusable toolchain, *occurred*: the development PC had no C++ compiler or CMake, and installing Visual Studio was not possible. The contingency was to use portable toolchains (Clang 21 via the Zig distribution, later MinGW-w64 GCC 16, and CMake and Ninja from Python packages), which cost roughly half a day rather than derailing the schedule. **R8**, unreliable internet, also occurred: dependency downloads broke repeatedly. The mitigation (pinning every dependency by version and SHA-256, and building from a local cache through CMake's `FETCHCONTENT_SOURCE_DIR_*` variables) made the build reproducible as well as robust. Building with a second compiler because of R1 had an unplanned benefit: GCC's stricter warnings found a defect (DEF-03) that Clang missed and that would otherwise have broken the Linux CI build. New risks were added as they emerged, for example R12 (the school placing the database on a network share, which can corrupt SQLite files) and R13 (CI not yet run on GitHub).

4. Requirements Engineering

4.1 Stakeholders

Nine stakeholders were identified and analysed by influence and interest (Appendix; `stakeholder-analysis.md`). The principal (sponsor), HoDs and lecturers have high influence or interest and were "managed closely". The student representative speaks for the largest user group but has low formal influence, which creates an ethical duty to make sure students' interests are not overridden simply because they carry less power. The IT officer owns the non-functional constraints. Parents and the examinations regulator are indirect stakeholders with interests in privacy and in the integrity of assessment records.

4.2 Conflicts and their resolution

The stakeholder statements contain seven conflicts. The four most significant, and the reasoning behind each resolution, are summarised here.

C1: when students see marks. The principal requires that students see no mark until the HoD has checked it; a lecturer wants students to see marks as soon as they are marked. The principal's rule was adopted because the principal is the sponsor, the HoD independently asks for the same control, and moderation protects both fairness and the school's reputation. Simply ruling against the lecturer would miss the *need* behind the statement, which is speed. The design therefore attacks the delay rather than the control: the HoD is notified the moment a mark awaits moderation, the moderation queue loads in well under two seconds even with 800 scripts, a whole assignment is released in one action, and the lecturer can see that a mark is "awaiting moderation".

C2: late work. The HoD wants late work penalised by 10 % of the available mark per day for three days and zero after that; the student representative insists that late work "should not simply be rejected". These statements conflict only if a zero is treated as a rejection. The resolution separates *acceptance* from *scoring*: late work is always accepted, stored and marked with written feedback, and the penalty affects only the final mark. A learner who submits five days late still receives feedback and a record that the work was done.

C3: extensions against penalties. Lecturers want to give extensions for good reasons, while the HoD wants penalties applied consistently. An extension therefore moves the deadline that the penalty is measured from, requires a written reason, and cannot be granted once the mark has been moderated, so it cannot be used to undo a moderation decision.

C6: security against speed on old hardware. Memory-hard password hashing is slow by design, yet the IT officer requires quick responses on old computers. Argon2id is used at the OWASP minimum parameters (19 MiB, two iterations) rather than the library's heavier defaults. The algorithm stays memory-hard while login stays around 0.1 s. Because the parameters are stored in each hash, they can be raised later without resetting anyone's password.

Twelve ambiguities were also resolved and confirmed at Gate 1. The most consequential concerned the meaning of "a day late". It was defined as **each started 24-hour period** after the student's effective deadline: one second late is one day, exactly 24 hours is one day, and 24 hours and one second is two days. The alternatives, calendar days and rounding to the nearest day, were rejected. Calendar days make the penalty depend on the time of day of the deadline, and rounding gives up to twelve hours of lateness for free. "10 % of the available mark" was interpreted as 10 % of the *maximum* mark, so the deduction is the same for every student at the same lateness. The deduction is capped so that a mark never becomes negative.

4.3 Software Requirements Specification

The SRS follows the structure of ISO/IEC/IEEE 29148:2018: introduction, overall description (user classes, environment, constraints, assumptions), specific requirements, and verification. It contains **21 functional requirements** (FR-01 to FR-21; FR-22 was added by CR1), **12 non-functional requirements** and **12 business rules**, each uniquely identified, MoSCoW-prioritised and tied to a verification method. Table 1 shows a representative subset.

ID	Requirement (abridged)	Priority	Verified by
FR-02	Each user can perform only the actions of their role (access matrix)	Must	52-case permission test, ST17
FR-11	Resubmission up to the effective deadline while unmarked; all versions kept	Must	SubmissionTest, ST02, ST04
FR-14	Lecturer enters raw mark and mandatory feedback; system applies late penalty	Must	29 penalty/lateness cases, ST08
FR-15	Every mark change audited: who, when, old, new, reason	Must	Audit trail tests
FR-17	HoD releases moderated marks; unmoderated marks stay hidden	Must	ModerationTest, ST11, ST15
NFR-01	Login within 2 s with 800 students stored (production Argon2id)	Must	Performance test: 153 ms
NFR-04	Passwords only as salted Argon2id, m >= 19 MiB, t >= 2, 16-byte salt	Must	Hasher tests, schema CHECK
NFR-06	100 % of SQL with user data uses bound parameters	Must	Inspection + injection tests
NFR-08	No invalid input or DB error ends the program	Must	10 exception-flow system tests
NFR-11	Business-logic coverage >= 70 %; no function CCN > 15	Should	gcovr 95.9 %, lizard max 14

Table 1 - Extract from the SRS (full SRS: `docs/04-requirements/srs.md`).

Every non-functional requirement was made measurable in the way the brief asks. "The system should respond quickly even with all 800 students on it" became three requirements with numbers, a data set and a reference build: NFR-01 to NFR-03. "Passwords must be protected" became two requirements (NFR-04 and NFR-05) naming the algorithm, its minimum parameters, the password rules, the lockout threshold and duration, and the requirement that a failed login does not reveal whether a username exists. Each can be checked by a test that either passes or fails.

4.4 Use case model

The use case diagram (Figure 3) covers the four actors. Generalisation from an abstract *User* actor carries the use cases shared by every role (log in, change password, view notifications), which avoids four identical sets of associations. «include» factors out mandatory reused behaviour: *Determine effective deadline* is needed to submit, to list assignments and to apply penalties, and *Record mark audit entry* is needed by both marking and moderation. «extend» models

optional behaviour inserted at an extension point under a condition: resubmission and late submission extend *Submit assignment*, re-marking extends *Mark submission*, and adjusting a mark extends *Moderate and release marks*.

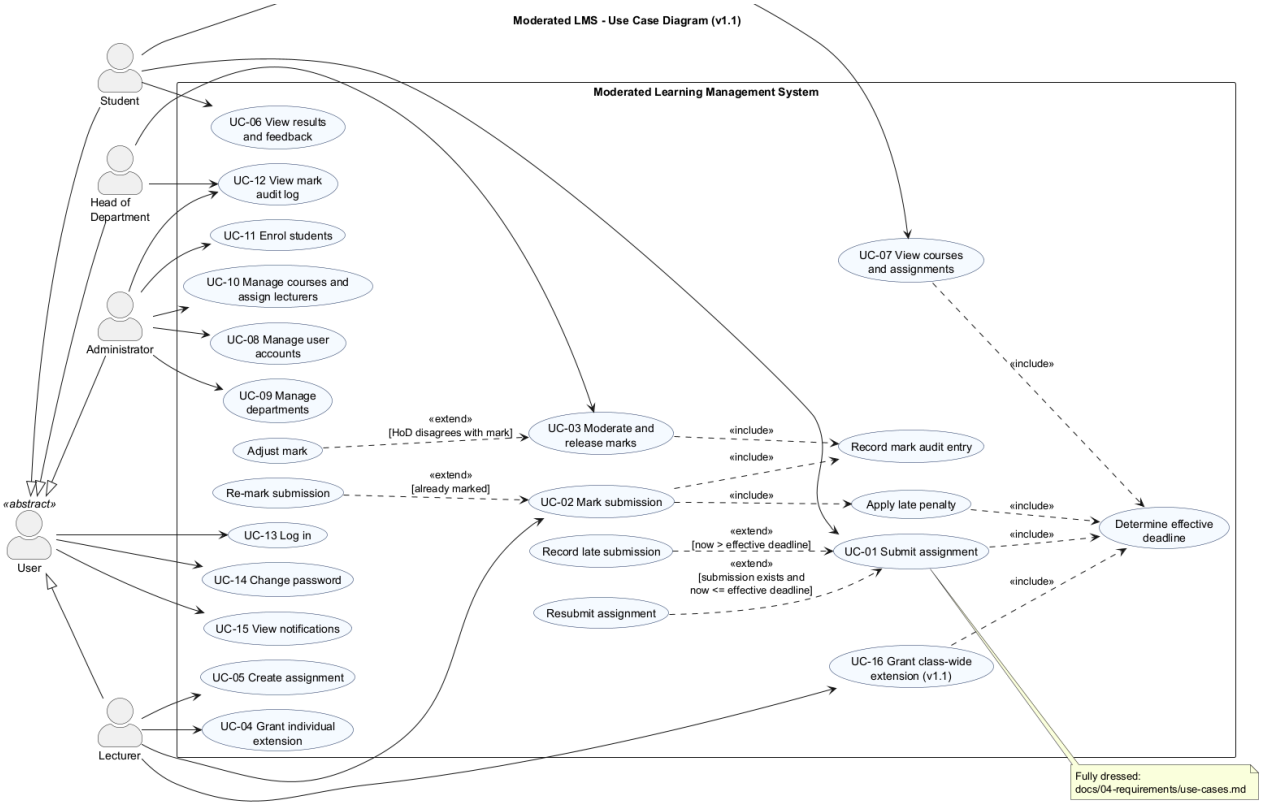


Figure 3 - Use case diagram (v1.1)

Three use cases were specified in fully dressed form: UC-01 *Submit assignment*, UC-02 *Mark submission* and UC-03 *Moderate and release marks*. UC-01 is reproduced below; the other two are in Appendix E. The alternative and exception flows proved to be the most valuable part of the specifications, because each became a system test (section 7).

UC-01 Submit assignment	
Primary actor	Student
Preconditions	The student is logged in and enrolled on the course; the assignment exists
Success postconditions	Submission stored as SUBMITTED or LATE with time and version; version recorded in the history; course lecturers notified
Failure postcondition	No stored data changes
Main flow	1. System lists the student's assignments with effective deadlines. 2. Student enters the assignment ID. 3. Student types the work, or `@path` to a file. 4. System checks enrolment and that the work is non-empty and at most 100 000 characters. 5. System determines the effective deadline. 6. No earlier submission exists and the deadline has not passed. 7. Submission stored as SUBMITTED, version 1, in one transaction. 8. Lecturers notified. 9. Student told until when resubmission is possible.
Alternative flows	A1 Resubmission before the deadline: work replaced, version incremented. A2 First submission after the deadline: stored as LATE, penalty rule shown. A3 File submission: regular file, at most 100 KB, never the LMS database.
Exception flows	E1 Resubmission after the deadline refused, original kept. E2 Already marked. E3 Not enrolled. E4 Unknown assignment. E5 Empty or oversized work. E6 Database error rolled back. E7 Input closed: clean exit.
Business rules	BR-03 (effective deadline), BR-04 (late work never rejected), BR-05 (resubmission window)

Table 2 - Fully dressed use case UC-01 (UC-02 and UC-03 in Appendix E).

5. Software Design

5.1 Architecture

A **strict layered architecture** was chosen (Figure 4). The presentation layer (console menus) calls only the business-logic layer. The business-logic layer (domain model, services, policies, security) depends only on *interfaces* of the data-access layer. The data-access layer (repositories, prepared statements, migrations) is the only code that contains SQL. The layers are separate CMake targets (`lms_ui` and `lms_core`), so the dependency rules are enforced by the build as well as by convention.

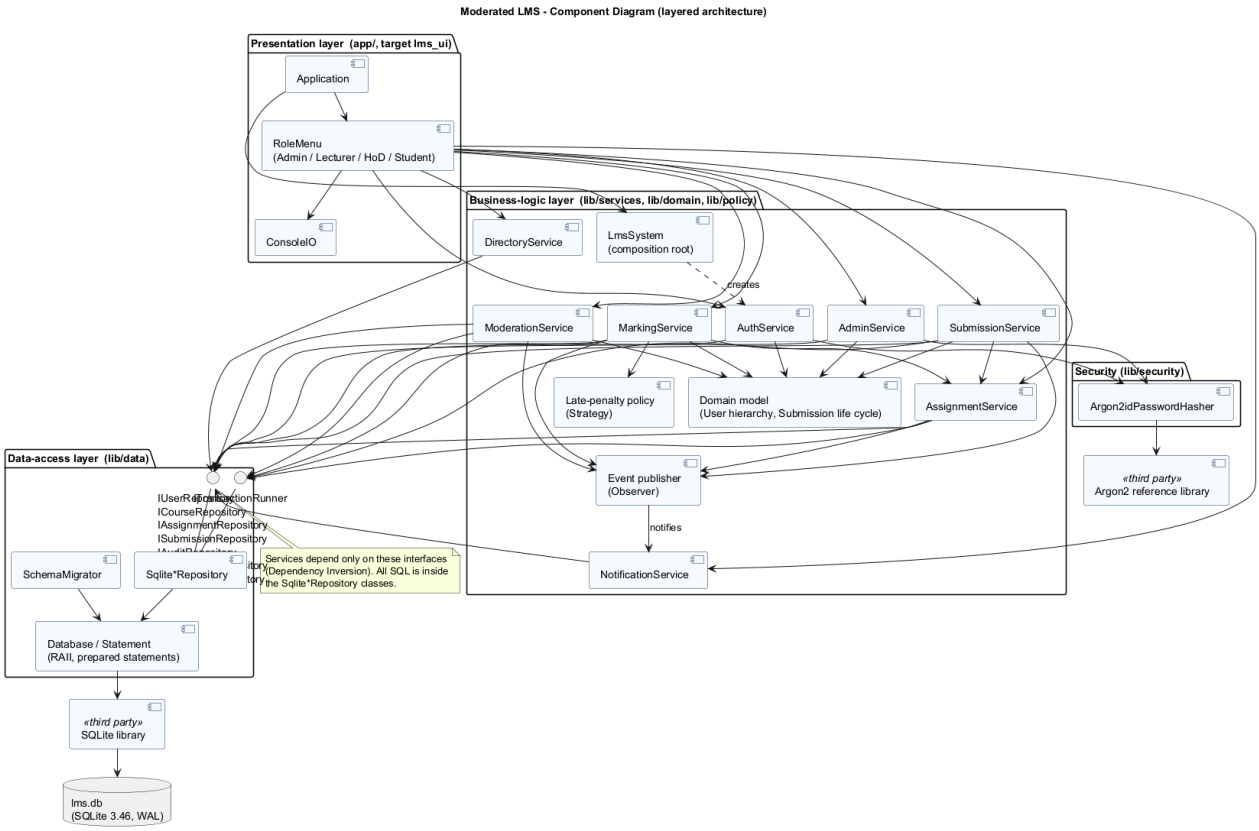


Figure 4 - Component diagram

The choice follows from the quality attributes that dominate this system. *Integrity* (moderation, audit) and *testability* (deadline and penalty logic) require business rules to live in one place that can be exercised without typing into a console; 278 automated tests run against that layer in about a second. *Maintainability* matters because a change request was expected, and the layers localise change. The data layer can be replaced, which matters because the most likely future requirement, several computers at once, would need a database server. The console could also be replaced by the "simple desktop GUI" the brief allows without touching the rules.

Three alternatives were considered and rejected. **Model-View-Controller** is designed for interfaces with several views that must stay synchronised with a changing model, as in GUIs and web applications. A console shows one screen at a time and never observes the model, so MVC's machinery would add controllers that duplicate the use-case services without any benefit; the part of MVC that matters here, keeping presentation separate from logic, is delivered by the layers. A **client-server** architecture would support concurrent users but violates the offline and old-hardware constraints and adds server administration the school cannot provide; it is recorded as future work. A single-module design with menus calling SQL directly would have been quickest to write, but it scatters the rules across user-interface code and makes them untestable in isolation. The maintenance phase later confirmed the choice (section 10).

5.2 Class design, patterns and SOLID

The design class diagram is presented as an overview plus three detailed views (Figures 5 to 7), because a single diagram with every member was about 9 700 pixels wide and unreadable. The diagrams show visibility, types, multiplicities, inheritance, interfaces and pattern roles.

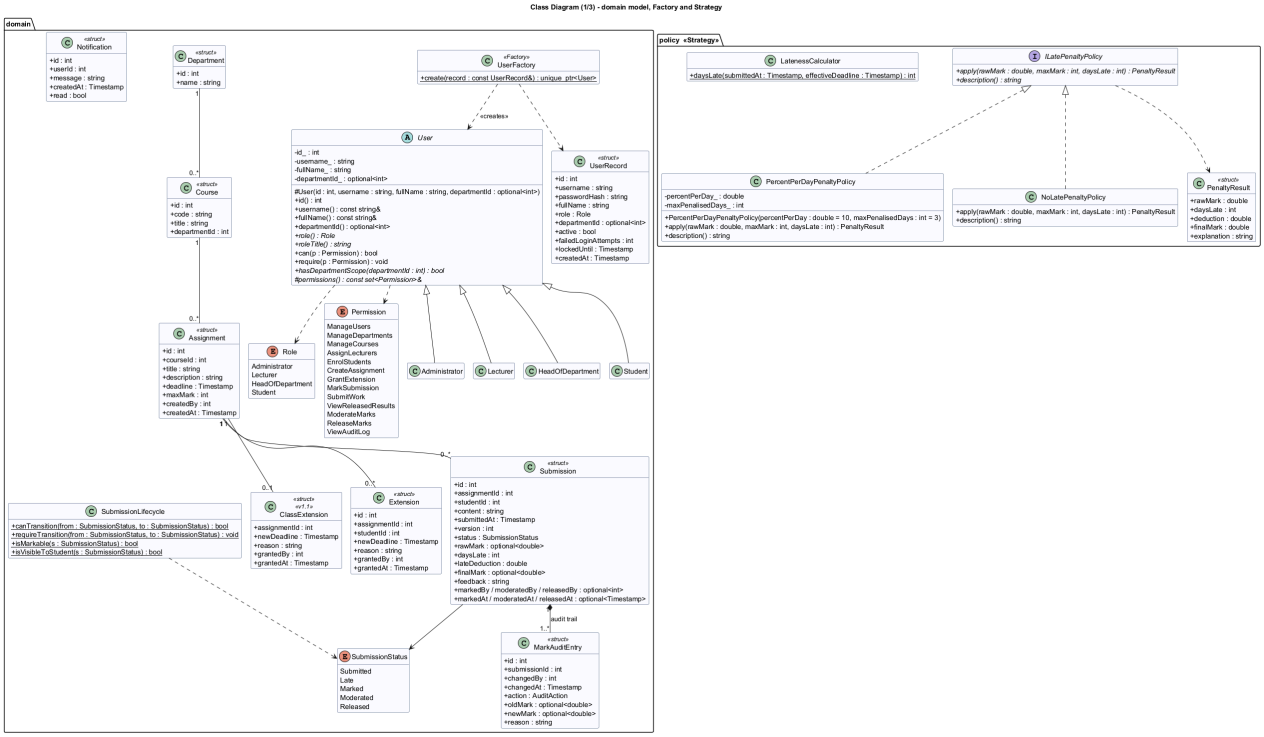


Figure 5 - Class diagram view 1: domain model, Factory and Strategy

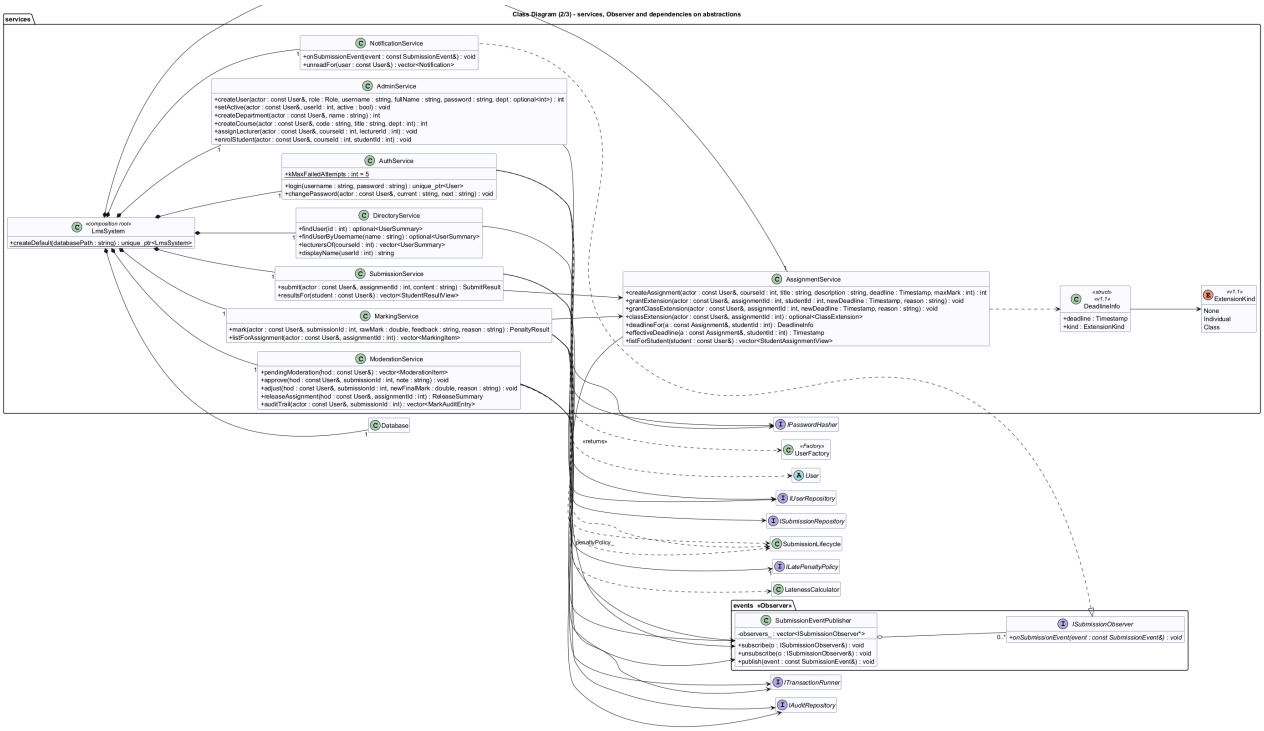


Figure 6 - Class diagram view 2: services and Observer

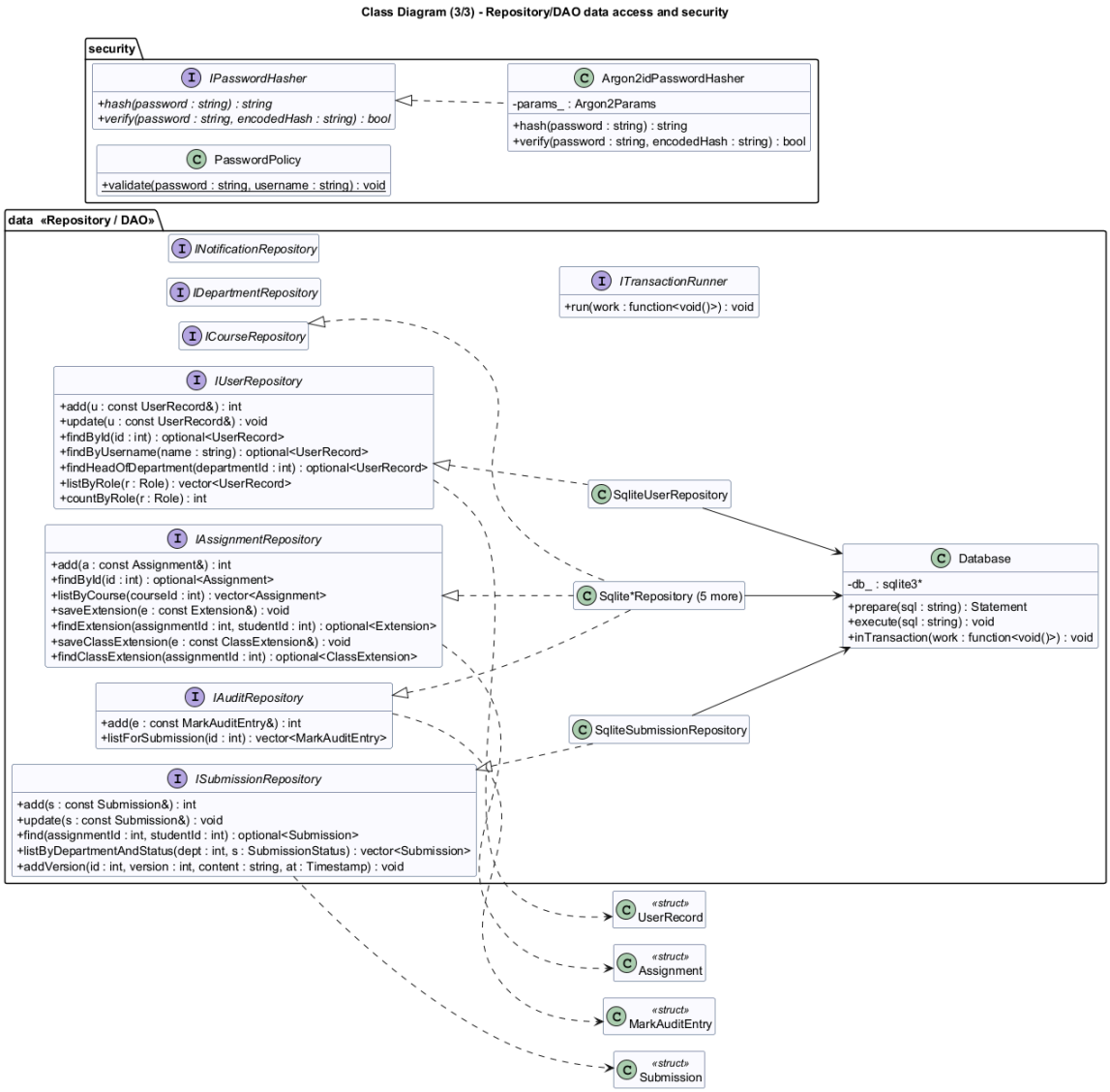


Figure 7 - Class diagram view 3: Repository/DAO and security

`User` is an abstract class with pure virtual functions for the role, its title, its permission set and its *department scope*. The four roles are `final` subclasses. Behaviour that differs by role is dispatched polymorphically rather than through conditionals on a role code. For example, `hasDepartmentScope(d)` returns true for every department for an Administrator, only the HoD's own department for a HoD, and false for everyone else, and the moderation service calls this without knowing which subclass it holds.

Five patterns are used, each chosen for a specific reason:

- **Strategy** for the late-penalty rule (`ILatePenaltyPolicy`, implemented by `PercentPerDayPenaltyPolicy(10 %, 3 days)` and `NoLatePenaltyPolicy`). The penalty was the most contested requirement and the one most likely to change. With a strategy the school can alter it, or waive it for a course, without editing the marking service, and a test swaps the strategy to prove that it is injectable.
- **Repository/DAO** for data access (seven small interfaces with SQLite implementations). It confines SQL to one file that can be inspected for injection safety, lets services be tested against an in-memory database, and would make a move to a server database local to one layer.
- **Factory Method** (`UserFactory`) is the single place that maps a stored role code to the right subclass; the authentication service returns a `unique_ptr<User>` without knowing the subclasses.
- **Observer** (`SubmissionEventPublisher` and `ISubmissionObserver`) decouples the services from notifications. Notifications were a Could requirement: they can be removed, or replaced by SMS later, without touching any service.
- **Template Method** (`RoleMenu::run`) fixes the skeleton of every session (greeting, notifications, menu loop and, above all, error handling) once, so every role gets identical robustness. Each subclass supplies only its title and items.

A composition root (`LmsSystem`) is the only place where concrete classes are chosen, which is what makes dependency injection work in the tests.

The design follows the **SOLID** principles (Martin, 2017). *Single responsibility:* `LatenessCalculator` only counts days late, the policy only turns days into a deduction, and `MarkingService` only orchestrates. *Open/closed:* new penalty rules, observers and schema changes are new classes or files, not edits. *Liskov substitution:* every policy and every `User` subclass is tested through its base type. *Interface segregation:* seven narrow repository interfaces rather than one "database" interface, and `IClock`, `IPasswordHasher` and `ITransactionRunner` each have one or two methods. *Dependency inversion:* services depend on

abstractions that the data layer implements. One compromise was accepted deliberately: services receive the whole bundle of seven repository references even when they use three or four, trading a little interface segregation for short constructors. It is documented rather than hidden.

5.3 Behavioural models

The **state machine** (Figure 8) defines the submission life cycle: SUBMITTED or LATE, then MARKED, MODERATED and finally RELEASED. The guards map to business rules: resubmission is allowed only while the effective deadline has not passed, a re-mark needs a reason, and RELEASED is final. An extension can move a LATE submission back to SUBMITTED. The table of legal transitions is implemented in exactly one function and tested exhaustively (all 25 from/to pairs).

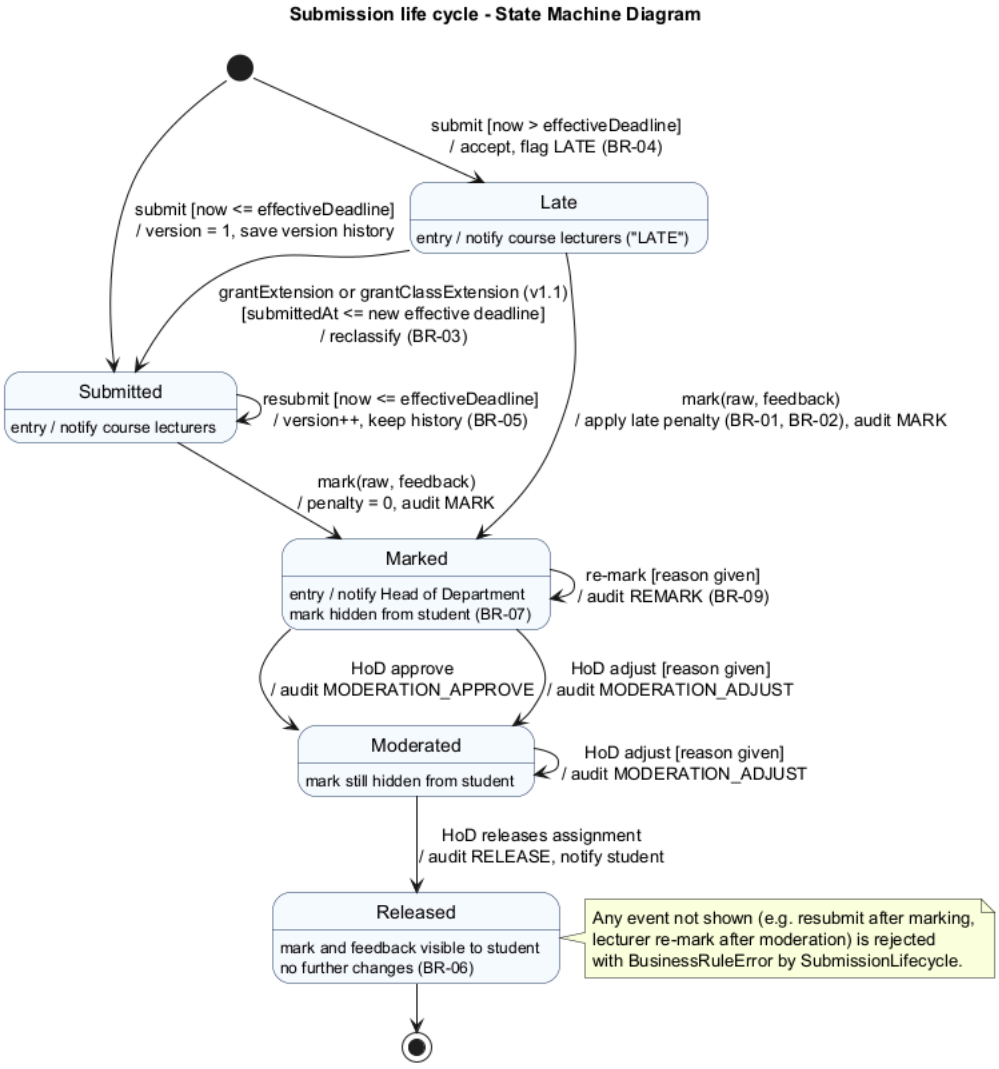


Figure 8 - State machine diagram for a submission

The **activity diagram** with swimlanes for Student, System, Lecturer and HoD (Figure 9) shows the end-to-end process from submission to released mark, including the resubmission and late branches, the optional extension and the moderation loop. The two **sequence diagrams** (Figures 10 and 11) show *Submit assignment*, with alternative fragments for the first, resubmitted, late and refused paths, and *Moderate and release marks*, with approve and adjust alternatives and the transactional release loop. They were used directly as the design of the service methods and as the basis for integration tests.

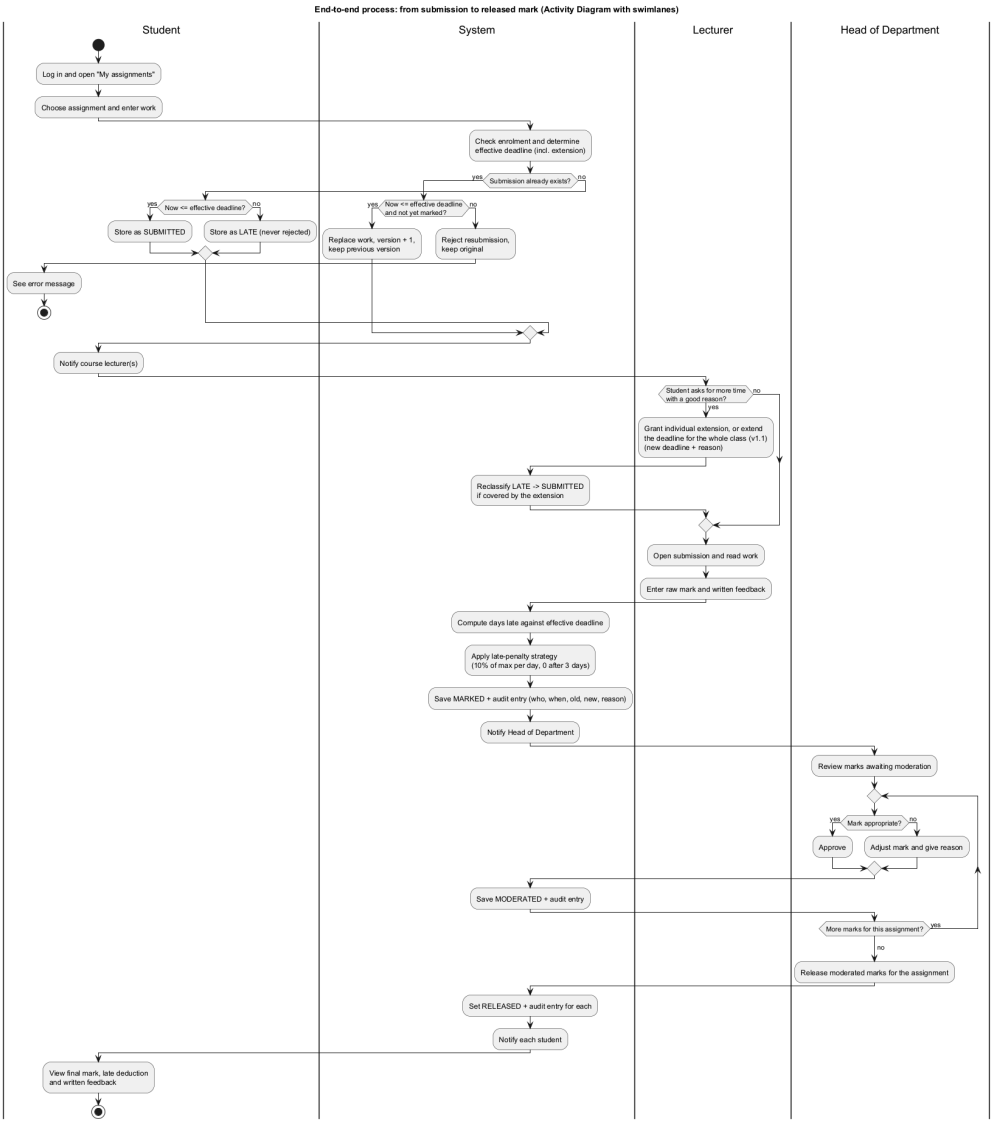


Figure 9 - Activity diagram: from submission to released mark

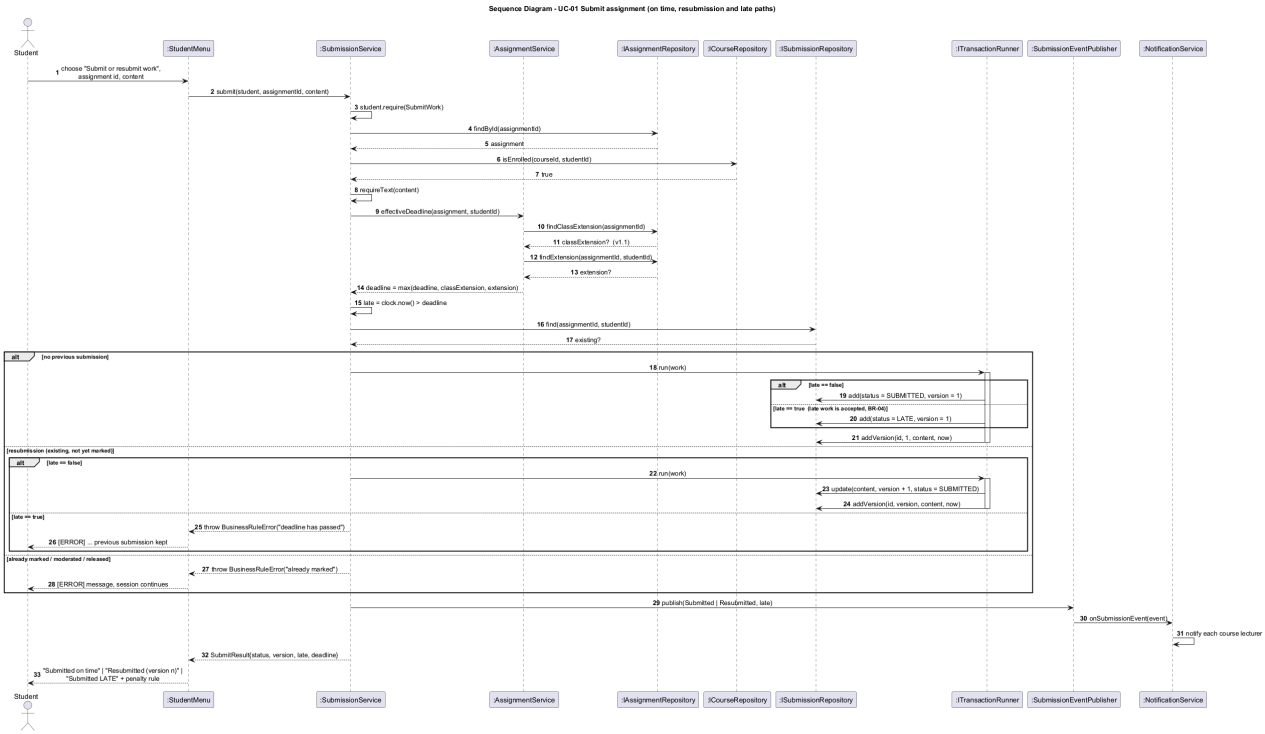


Figure 10 - Sequence diagram: Submit assignment (on-time, resubmission and late paths)

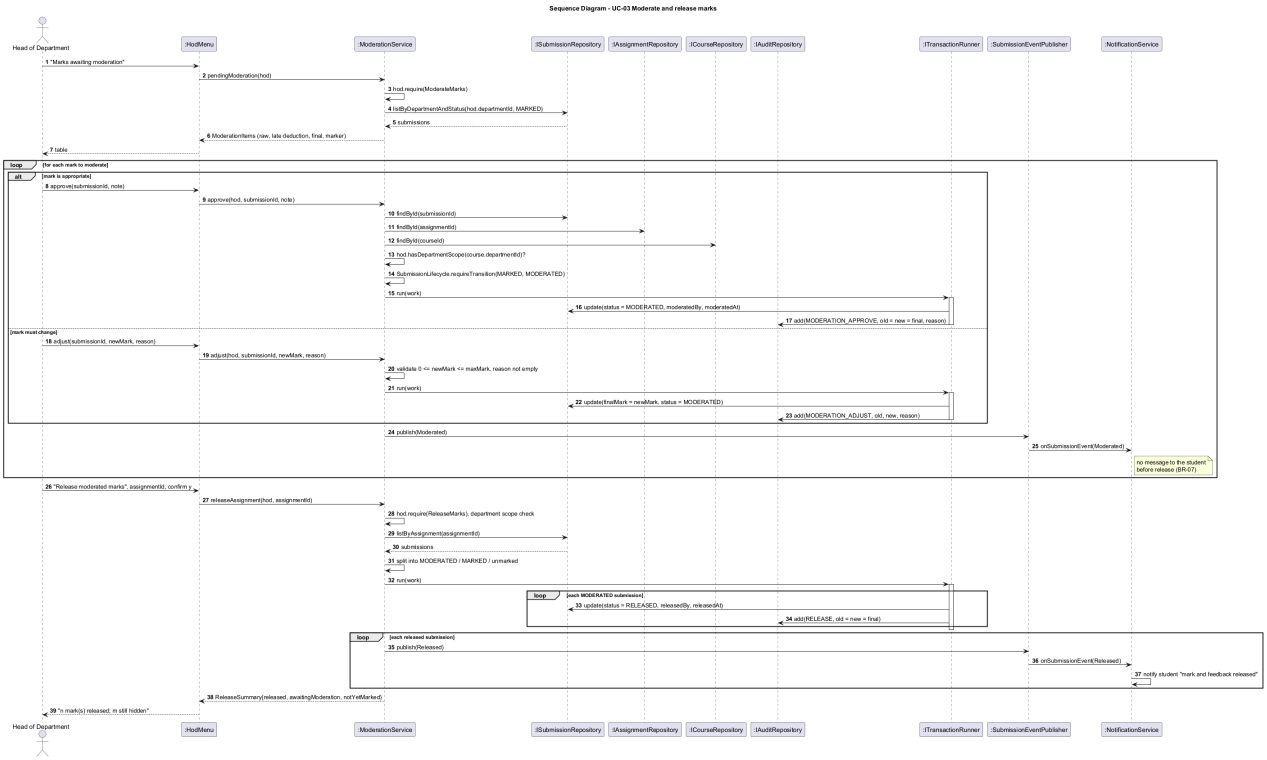


Figure 11 - Sequence diagram: Moderate and release marks

5.4 Database design

The ERD (Figure 12) has twelve tables in v1.1 and is normalised to third normal form. Many-to-many relationships (lecturers to courses, students to courses) are junction tables, and resubmission history is a separate table rather than a repeating group. No non-key attribute depends on another non-key attribute. The HoD of a department is not stored in `department`, because that would duplicate the user's role and department and could contradict it; a partial unique index instead enforces "at most one active HoD per department" directly on `app_user`.

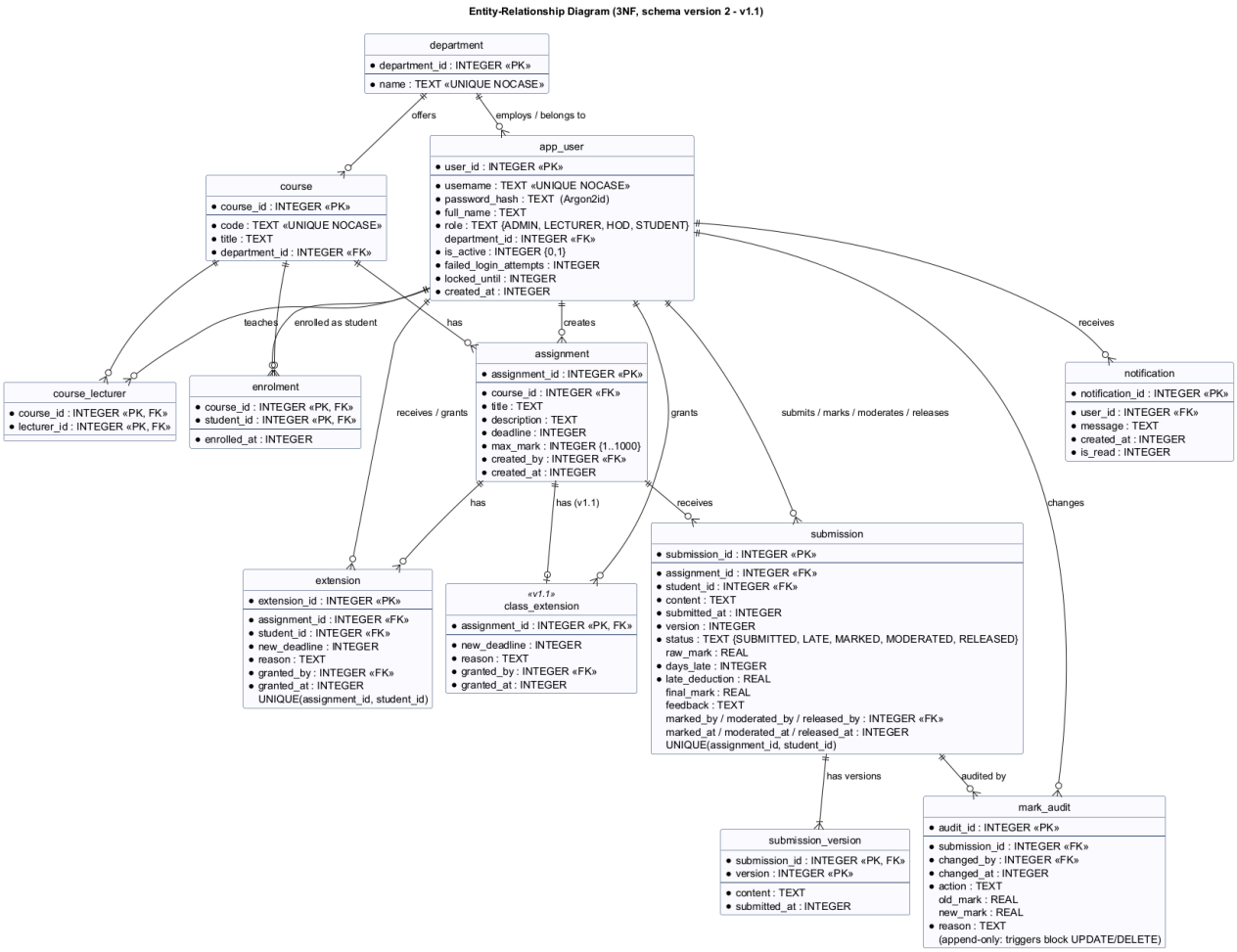


Figure 12 - Entity-relationship diagram (schema version 2)

One attribute was examined closely as a possible transitive dependency: `submission.final_mark`. At marking time it equals the raw mark minus the late deduction, so it looks derivable. It is stored anyway for two reasons. After moderation the HoD may set a different value, so it becomes an independent fact. And an assessment record must not change silently if the penalty policy changes later, so the stored `days_late`, `late_deduction` and `final_mark` are the recorded decision that the audit log refers to.

The data dictionary, the full schema and the class-to-table mapping are in the SDD. The main mapping decision is **single-table inheritance** for users: the four subclasses add behaviour but no data, so one table with a role discriminator avoids four identical tables and a join at every login. Security-sensitive columns (the password hash and lockout counters) exist only in the persistence object `UserRecord` and never reach the user interface, which receives a separate `UserSummary`. Constraints are enforced in the database as well as in code: CHECK constraints on roles, statuses, mark ranges and the Argon2id hash prefix; foreign keys; uniqueness; and two triggers that make the audit table append-only. The schema lives in numbered SQL files that CMake embeds into the executable, so the documentation, the tests and the program cannot use different schemas.

6. Implementation

6.1 Structure

The code comprises about 4 800 non-blank lines of production C++ and SQL and 2 000 lines of tests. `lib/` holds the domain, services, policy, security and data-access code (the `lms_core` library); `app/` holds the console presentation layer; `sql/` holds the migrations; `tests/` holds the four test levels. The project builds with CMake. SQLite 3.46.1, the Argon2 reference implementation and GoogleTest are fetched by pinned URL and SHA-256, so every build is reproducible, and they can come from a local cache for offline builds. A documented coding standard (naming, RAII, no raw `new`, const-correctness, exception rules, security rules, complexity limits) is enforced partly by the compiler warning set and the static analysers.

6.2 Key decisions

Time is an integer. All timestamps are 64-bit seconds since the Unix epoch, and local time is used only for display and input. Deadline comparisons and day counting are therefore exact integer arithmetic, with no time-zone or daylight-saving ambiguity, which directly addressed risk R5. The clock is injected through an `IClock` interface, so tests can place "now" exactly one second after a deadline.

The effective deadline is computed in exactly one place (`AssignmentService::deadlineFor`). Submission, marking and listing all ask this method. This single decision is what made the maintenance change cheap (section 10).

Lateness is evaluated at marking time, against the effective deadline *then*. An extension granted after a late submission, for example once a power cut has been confirmed, therefore removes the penalty automatically, and the service also reclassifies any unmarked LATE submission the extension now covers.

Errors are exceptions from one hierarchy (`ValidationError`, `AuthorizationError`, `BusinessRuleError`, `NotFoundError`, `DatabaseError`). Every service entry point checks, in order, the actor's permission, their course or department scope, the input, and the business rule. Only then does it write, inside a transaction. The Template Method in the menu catches `LmsError` in one place, reports it in plain language and continues the session. The input layer re-prompts on malformed numbers and dates, and end of input exits cleanly. System tests prove that none of the exception flows terminates the program.

6.3 Security

Passwords are hashed with **Argon2id**, the winner of the Password Hashing Competition, standardised in RFC 9106 and recommended first by OWASP. It uses a 16-byte salt from the operating system's cryptographically secure generator (`BCryptGenRandom` on Windows, `/dev/urandom` elsewhere). The reference library was preferred to alternatives on stated grounds. bcrypt truncates passwords at 72 bytes and is not memory-hard. PBKDF2 is not memory-hard, so GPU attacks are cheap, and using it would mean building OpenSSL offline on Windows. libsodium would have been equally acceptable but is heavier to build. The encoded hash stores its own parameters, so they can be strengthened later.

Accounts lock for 15 minutes after five consecutive failures. The login error message is identical whether the username or the password was wrong. After the self code review (section 8) the *timing* is identical too: for an unknown user the service verifies the password against a dummy hash, so response time does not reveal which usernames exist. Passwords are read without echo. All SQL uses prepared statements with bound parameters, so injection payloads used as usernames simply fail to match, as a test demonstrates. Every service call checks role and scope, so the menus are a convenience, not the security boundary.

6.4 Persistence

Each connection enables foreign keys, write-ahead logging and a busy timeout. Transactions use nested SAVEPOINTs, so a service that calls another can safely nest a unit of work. A mark and its audit entry are always written in the same transaction, which is what makes the claim "every change is audited" true even when a write fails midway. Migrations are applied in order with `PRAGMA user_version`, and a database created by a newer release is refused rather than silently corrupted.

7. Verification and Validation

7.1 Strategy

The test plan maps each V-Model level to the artefact it verifies. **Unit tests** verify the detailed design: lateness, penalties, the user hierarchy and permissions, the state machine, hashing, the password policy, date parsing and the event publisher, with no database. **Integration tests** verify the architecture: the services running against real SQLite repositories in memory, including transactions, migrations, constraint enforcement and the observer. **System tests** verify the SRS by driving the real console application with scripted keyboard input, one test per use-case flow, together with performance tests with 800 students. **Acceptance tests** validate stakeholder needs by running the packaged executable against a fresh demonstration database. Each level has entry and exit criteria; for example, integration testing starts only when unit tests pass and ends when all integration tests pass with coverage of at least 70 %.

7.2 Test design

Test cases were derived systematically (Myers, Sandler and Badgett, 2011). For **lateness**, the input domain was partitioned into on time, day 1, days 2-3 and more than three days, and tests sit on both sides of every boundary: one second early, exactly at the deadline, one second late, exactly 24 hours, 24 hours plus one second, and so on up to 72 hours plus one second. For **penalties**, partitions cover no penalty, the lowest and highest penalised days, the zero partition, the cap at the raw mark, the boundaries of the mark range and invalid values (above the maximum, negative, NaN). Similar boundary tables cover password length (7, 8, 128, 129), username length, maximum mark, submission and feedback sizes, the lockout threshold (four failures versus five) and the lockout duration (one second before and exactly at 15 minutes). **State-transition testing** covers all 25 transitions of the submission state machine, and a **decision table** covers all 52 role-permission combinations.

7.3 Results

Table 3 summarises the automated results for v1.0 (the v1.1 regression results are in section 10).

Level	Tests	Passed
Unit	145	145
Integration	92	92
System (ST01-ST27)	27	27
Performance (NFR-01..03)	5	5
Acceptance scenarios (AT-01..05)	4	4

Table 3 - Test results for release v1.0 (evidence in `docs/07-testing/reports`).

All functional tests pass with two compilers: GCC 16 with warnings as errors, and Clang 21. **Line coverage of the business-logic layer is 95.9 %** (function coverage 94.9 %), against a target of 70 %. The uncovered lines are mainly SQLite bind-failure paths that cannot be reached with valid statements and the production factory used only by the executable. Branch coverage (59 %) is reported but not targeted, because GCC counts the hidden exception-handling branches the compiler generates for every call that can throw. The performance requirements were met with wide margins: login 153 ms (limit 2 s), assignment list 2.6 ms, submission 1.9 ms, a moderation queue of 800 items in 108 ms, and releasing 800 marks in 338 ms.

The acceptance run produced an instructive observation. The demonstration assignment's deadline was exactly two days before the submission, which was made a few seconds later, so the work counted as three started days late and lost 30 %. This is exactly what BR-02 specifies, but users might not expect it, and the student manual now explains the rule with a worked table. One acceptance test, AT-06 (a first-time student completes a submission within two minutes using the manual), needs a real volunteer and has **not yet been executed**; this is stated in the test report rather than claimed.

7.4 Defects

Eight defects were logged, with severity, how each was found, its cause, the fix (commit) and the retest result (Appendix B). Three are worth discussing. **DEF-01** was found by a unit test: the penalty policy accepted NaN, because `raw < 0 || raw > max` is false for NaN. The fix rewrote the check in the positive form, and a coding-standard rule now requires it. Later, clang-tidy recommended "simplifying" that same expression back to the defective form. The suggestion was rejected with a documented suppression, a small example of why tool advice needs engineering judgement. **DEF-04** and **DEF-05** were security defects found by *review*, not testing: the login timing side channel, and a file submission path that could have submitted the LMS database itself, exposing all marks and password hashes to a lecturer. That two of the most serious defects escaped 260 tests supports keeping a review checklist alongside automation.

8. Configuration Management and Quality Assurance

8.1 Configuration management

Every configuration item is under Git: code, migrations, build scripts, CI definition, tool profiles and every document and diagram source. Third-party code is identified exactly (version and hash) rather than copied. The branching strategy is **GitHub Flow with release tags**. `main` is always releasable, and every change arrives from a short-lived `feature/`, `fix/` or `docs/` branch through a merge commit, so the history shows each unit of work (Figure 13 shows the shape). Git Flow was considered and rejected: permanent `develop` and `release` branches would double the merges for a single developer with two planned releases without adding any gate that CI and tags do not already provide. Commits follow a conventional format and cite requirement and defect IDs, for example `fix(policy): reject NaN raw marks in the penalty policy (DEF-01)`. Releases are annotated tags `v1.0` and `v1.1`, and the version number is defined once in CMake and propagated to the program and the package name.

```
*   Merge branch 'feature/cr1-class-extension'        (tag v1.1)
|\  docs(cr1) update affected documents / feat(cr1) class-wide extensions / docs(cr1) impact analysis
*   Merge branch 'docs/v1.0-deliverables'             (tag v1.0)
*   Merge branch 'fix/code-review-findings'   refactor: review, static-analysis and metrics fixes; DEF-03
*   Merge branch 'feature/ci'  ...  'feature/automated-tests' (fix DEF-01) ... 'feature/build-setup'
```

Figure 13 - Shape of the Git history (`git log --graph`).

The **CI pipeline** (GitHub Actions) runs on every push and pull request. It builds Debug and Release configurations on Ubuntu and Windows, with warnings as errors on Linux; runs all functional tests, and the performance tests in Release; fails if line coverage of `lib/` drops below 70 %; fails on any cppcheck finding; publishes the clang-tidy report; and, for version tags, builds a self-contained Windows ZIP with CPack and attaches it to a GitHub release. One honest limitation: the development PC had no GitHub remote during the project (risk R13), so every CI job was reproduced locally with the same compilers and flags. The actual GitHub Actions evidence is produced when the repository is pushed.

8.2 Quality assurance

Static analysis used two tools. cppcheck initially reported 16 style and performance findings. The genuine ones (names shadowing the menu's own `title()` method, a structure passed by value) were fixed; nine suggestions to replace readable loops with algorithms were consciously declined and suppressed with a reason. clang-tidy, with a broad profile, reported 59 findings. Most were fixed, including two real robustness defects: an unchecked return value from the time-conversion function and an optional value accessed without an explicit check. Two were kept with documented justifications: a `reinterpret_cast` required by SQLite's C API, and an empty catch that preserves the original exception during rollback. Both analysers now report zero findings.

Metrics were collected with lizard. The production code has 274 functions with an average cyclomatic complexity of 2.8 (McCabe, 1976). Two functions exceeded the limit of 15 set in the coding standard, the date parser (19) and the notification dispatcher (16); both were refactored into helpers, and the maximum is now 14, a flat switch that maps permissions to text. The key business functions (`submit`, `mark`, `daysLate`) stay below 10.

A **self code review** against a 16-point checklist (permission checks first, scope checks, bound SQL, no secrets in the UI, atomic mark-plus-audit, NaN-safe ranges, bounded input, timing-safe login, complexity, naming, consistency with the class diagram) found five issues that led to changes and four that were accepted with reasons. For example, an N+1 query pattern in the moderation queue was kept because it measured 108 ms for 800 items against a 2 s limit, and optimising it would have complicated the repository interface for no user-visible gain. The review also found that `DirectoryService` existed in the code but not in the class diagram; the diagram was corrected, which is what the brief's consistency requirement asks for.

9. Deployment

The deployment diagram (Figure 14) shows the intended configuration: one school computer running a single statically linked executable that contains the business logic, SQLite and Argon2. It works with one database file in write-ahead-log mode and a daily copy to a backup drive, and no network connection is required. Static linking of the C/C++ runtime (a build option used by the release job) means the school does not need to install a Visual C++ redistributable.

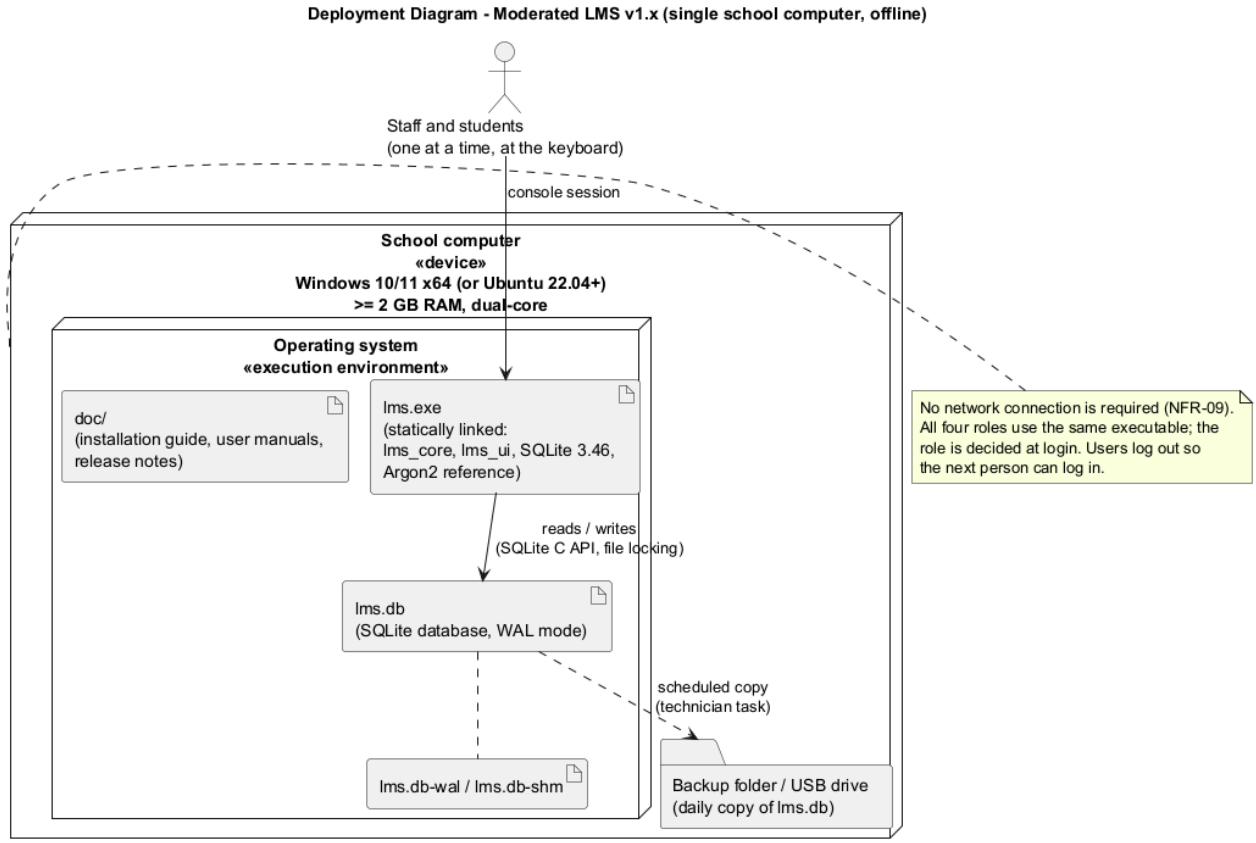


Figure 14 - Deployment diagram

The **installation guide** is written for a school technician rather than a developer. It gives numbered steps for extracting the ZIP, creating a data folder and a desktop shortcut, protecting the folder, creating the first administrator on first start, and training with a separate demonstration database. It explains backup and restore, including a Task Scheduler recipe, and in-place upgrades. It lists what must not be done (a network share, editing the database, changing the clock) and includes a troubleshooting table. A **user manual for each role** explains the menu options in the users' language, with worked examples of the late-penalty rule for students. **Release notes** list features, security measures, quality figures and known limitations. The release was tagged `v1.0`, and CPack produces `ModeratedLMS-1.x.x-Windows.zip` containing the executable and the documentation.

10. Maintenance: Change Request CR1

After release the school requested: *"A lecturer should be able to extend the deadline for a whole class, not just an individual student, and extensions should be shown on the student's assignment list."*

Classification. The change is **perfective maintenance** in the sense of Lientz and Swanson (1980) and ISO/IEC/IEEE 14764. It is not corrective, because v1.0 met its specification (FR-08 asked only for individual extensions) and no defect is involved. It is not adaptive, because nothing in the environment changed. It is not preventive, because its purpose is not to ease future maintenance. Users are asking for new functionality and better presentation of existing information after using the system. It has a small adaptive element: existing v1.0 databases must be upgraded in place without losing data.

Clarification and impact analysis. Before any code was written, the request was clarified. A "whole class" is everyone enrolled on the course. There is one class extension per assignment, and granting it again replaces the previous one. The student's effective deadline becomes the *latest* of the original deadline, the class extension and their own extension, so neither kind can shorten the other. The change is refused once moderation has begun, consistent with the rule for individual extensions. The impact analysis then listed every affected artefact: - requirements FR-09 and FR-19, business rule BR-03 and the new FR-22; - use cases (new UC-16) and all class views, the ERD, the state machine, the activity diagram and the submit sequence diagram (the component and deployment diagrams are unaffected); - about a dozen named source files; - one new table, `class_extension`, as an additive migration; - tests: the analysis predicted that exactly one existing system test (ST22) would fail because it checked the v1.0 display text "(extended)", and it listed the new tests needed.

Implementation. The change was made on its own branch, in three commits (analysis, code and tests, updated documents), and released as `v1.1`. The layered design and the single effective-deadline method paid off: late status, penalties, resubmission windows and reclassification all started respecting class extensions **without any change to the submission or marking services**. The lecturer menu gained one option, and the student list gained *Original deadline* and *Extension* columns.

Regression testing. The existing suite was first run unchanged against the new code: 263 of 264 passed, and the single failure was ST22, exactly as predicted, for an intended display change rather than a behavioural regression. After that expectation was updated, 14 new tests were added: 11 integration tests for the rules above (including both orders of individual and class extensions, boundary values, reclassification, refusal after moderation, penalty removal and notifications); a test that builds a genuine v1.0 schema with data and verifies the upgrade to schema version 2 without loss; and two system tests. The full suite

of **278 tests passed** on both compilers, the performance and acceptance tests passed (including a new acceptance scenario for the class extension), coverage rose to 96.1 %, and the static analysers reported no findings.

11. Professional, Legal and Ethical Issues

Data protection. The system processes personal information about learners, most of them minors: names, their work, marks and feedback. The Lesotho Data Protection Act 2011 requires such information to be processed lawfully, for a specific purpose, adequately and not excessively, accurately, securely, and not kept longer than necessary, and it gives data subjects rights of access and correction. The school is the data controller. The design supports these duties by construction. It stores no more than assessment needs: no identity numbers, addresses or birth dates. Access is restricted by role and scope in every service call, and students see only their own work. Everything stays on the school's computer, so nothing is transferred across borders. Records are deactivated rather than deleted, so decisions can be checked on appeal. Some duties can only be met by the school, and the report says so rather than implying the software discharges them: a retention period, archiving data of learners who have left, physical security of the PC, encryption of backup drives, and informing parents. Two technical limitations are declared openly: the database file itself is not encrypted, and passwords exist briefly in ordinary memory during login. Both are proportionate risks for a single offline computer, and both have known remedies for a future version.

Security of assessment records. Marks affect progression and reputations, so the system treats them as evidence. Separation of duties (the lecturer marks, the HoD moderates and releases, neither can do the other's step), the state machine that makes released marks final, and an audit log that the database itself refuses to update or delete together mean that a disputed mark can be traced to a person, a time and a stated reason.

Accessibility. A plain-text console is, perhaps surprisingly, a good basis for accessibility. Screen readers such as NVDA and Narrator read it, nothing is conveyed by colour alone, users can enlarge the text through the console's own settings, and the interface is keyboard-only with no time pressure. Cognitive load is limited by short numbered menus and error messages that say what to do. Individual extensions give learners who need extra time a recorded, private mechanism. The limits are acknowledged: long tables are tedious to hear read aloud, and the usability acceptance test with real users is still outstanding.

Fairness of the late penalty. The rule is the HoD's policy. The software's ethical obligation is to apply it consistently, transparently and without hidden harshness. One implementation, tested at every boundary, replaces manual arithmetic that varied between lecturers. The rule is shown to students at the moment they submit late and is explained with examples in their manual, so no one is penalised by a rule they could not know. The deduction is proportional to the maximum mark and capped at the student's mark. Late work is always marked with feedback. The "started 24-hour period" definition has a harsh edge (one second past 24 hours counts as two days), but the alternative rewards lateness, and the chosen rule is at least predictable. In a context of unreliable electricity and shared computers, lateness is not always the learner's fault, so the design deliberately makes compassion easy but visible: extensions and moderation adjustments are available, but each requires a reason and leaves a trace. Because the rule is a Strategy, the school can adopt a gentler policy without rewriting the system.

Professional conduct. Following the ACM/IEEE Software Engineering Code of Ethics, the documents report what was not verified as clearly as what was: the pending usability test, CI not yet run on GitHub, and the gap between the plan and the actual timeline. All third-party components are used within their licences and are attributed. Where AI assistance was used in producing code or documents, the student remains responsible for understanding and being able to explain every part at the viva, and commits made with assistance record it.

12. Conclusion

The project delivered a working, tested and documented moderated LMS that meets every Must requirement, together with the complete chain of artefacts the Waterfall and V-Model processes require: plan, specification, design, code, tests, quality records, deployment material and a maintained second release. The strongest evidence that the engineering decisions were sound comes from the maintenance phase. A change that touched the central concept of the deadline was absorbed by one service method and one additive table, while the existing tests confirmed nothing else moved.

The project also taught lessons that should be stated critically. First, **resolving conflicts by looking for the need behind each statement** (speed for the lecturer, not rejecting work for the students) produced better requirements than choosing a winner. Second, **tests, reviews and tools find different defects**: a unit test found the NaN defect, a review found both security defects, a second compiler found the build defect, and a static analyser both found real problems and once gave wrong advice. None of these techniques would have been sufficient alone. Third, **Waterfall's rigidity was mitigated rather than eliminated.** Explicit gates and traceability kept the artefacts consistent, but the real timeline was more compressed than the plan, and the documents record that honestly rather than reconstructing an idealised history.

The main limitations are the single-computer deployment, the unencrypted database file, the outstanding usability test and CI that has not yet run on GitHub. Future work, in order of value to the school, would be: a client-server version for several computers; encryption at rest (SQLCipher); a controlled procedure for correcting released marks; and export of results for reporting to parents and the ministry.

13. References

ACM/IEEE-CS Joint Task Force (1999) *Software Engineering Code of Ethics and Professional Practice*, version 5.2. New York: ACM.

Biryukov, A., Dinu, D., Khovratovich, D. and Josefsson, S. (2021) *RFC 9106: Argon2 Memory-Hard Function for Password Hashing and Proof-of-Work Applications*. Internet Engineering Task Force. Available at: https://www.rfc-editor.org/rfc/rfc9106 (Accessed: 28 September 2026).

Cockburn, A. (2001) *Writing Effective Use Cases*. Boston: Addison-Wesley.

Elmasri, R. and Navathe, S.B. (2016) *Fundamentals of Database Systems*. 7th edn. Harlow: Pearson.

Forsberg, K. and Mooz, H. (1991) 'The relationship of system engineering to the project cycle', *Proceedings of the First Annual Symposium of the National Council on Systems Engineering*, pp. 57-65.

Fowler, M. (2002) *Patterns of Enterprise Application Architecture*. Boston: Addison-Wesley.

Gamma, E., Helm, R., Johnson, R. and Vlissides, J. (1994) *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA: Addison-Wesley.

Government of Lesotho (2011) *Data Protection Act, 2011 (Act No. 5 of 2011)*. Maseru: Government Printer.

ISO/IEC/IEEE (2018) *ISO/IEC/IEEE 29148:2018 Systems and software engineering - Life cycle processes - Requirements engineering*. Geneva: International Organization for Standardization.

ISO/IEC/IEEE (2022) *ISO/IEC/IEEE 14764:2022 Software engineering - Software life cycle processes - Maintenance*. Geneva: International Organization for Standardization.

Lientz, B.P. and Swanson, E.B. (1980) *Software Maintenance Management*. Reading, MA: Addison-Wesley.

Martin, R.C. (2017) *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Boston: Prentice Hall.

McCabe, T.J. (1976) 'A complexity measure', *IEEE Transactions on Software Engineering*, SE-2(4), pp. 308-320.

Myers, G.J., Sandler, C. and Badgett, T. (2011) *The Art of Software Testing*. 3rd edn. Hoboken, NJ: Wiley.

OWASP Foundation (2024) *Password Storage Cheat Sheet*. Available at: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html (Accessed: 28 September 2026).

Royce, W.W. (1970) 'Managing the development of large software systems', *Proceedings of IEEE WESCON*, 26, pp. 1-9.

Sommerville, I. (2016) *Software Engineering*. 10th edn. Harlow: Pearson.

SQLite Consortium (2024) *Appropriate Uses For SQLite*. Available at: https://www.sqlite.org/whentouse.html (Accessed: 28 September 2026).

Stroustrup, B. and Sutter, H. (eds.) (2024) *C++ Core Guidelines*. Available at: https://isocpp.github.io/CppCoreGuidelines/ (Accessed: 28 September 2026).

W3C (2023) *Web Content Accessibility Guidelines (WCAG) 2.2*. Available at: https://www.w3.org/TR/WCAG22/ (Accessed: 28 September 2026).

14. Appendices

Appendix A - Requirements traceability matrix

Forward trace: requirement -> use case -> design elements (classes, diagrams, tables) -> source files -> tests. Every functional requirement is implemented and has at least one passing automated test. Paths are relative to the repository root; `S:` = `lib/src/services/`, `U:` = `app/src/`.

Functional requirements

FR	Use case	Classes	Diagrams	Tables	Source files	Te
FR-01 Login	UC-13	`AuthService`, `IPasswordHasher`, `UserFactory`	class (2), component	app_user	S:AuthService.cpp, lib/src/security/PasswordHasher.cpp, U:Application.cpp	AuthTest.LoginReturnsPolymorphicI UsernameIsCaseInsensitiveAndTrim WrongPasswordAndUnknownUserC EmptyCredentialsRejected; ST16
FR-02 Role-based access	UC-13 (all)	`User` + 4 subclasses, `Permission`, `RoleMenu` subclasses	class (1), use case	app_user.role	lib/src/domain/User.cpp, Role.cpp, U:RoleMenu.cpp	UT_USR/PermissionMatrix (52), UT_USR.DepartmentScopeIsPolymc AdminTest.OnlyAdministratorsMana SubmissionTest.OnlyStudentsSubmi ModerationTest.OtherDepartmentA ST17
FR-03 Manage accounts	UC-08	`AdminService`, `PasswordPolicy`, `UserSummary`	class (2)	app_user	S:AdminService.cpp, U:AdminMenu.cpp	AdminTest.CreatesUsersOfEveryRol CannotDeactivateOwnAccount, Rea ListUsersHidesPasswordHashes; Aut AdministratorResetUnlocksAccount
FR-04 Departments	UC-09	`AdminService`, `Department`	class (1, 2), ERD	department	S:AdminService.cpp	AdminTest.DepartmentAndCourseN
FR-05 Courses, lecturers	UC-10	`AdminService`, `Course`, `ICourseRepository`	class, ERD	course, course_lecturer	S:AdminService.cpp, lib/src/data/SqliteRepositories.cpp	AdminTest.LecturerOnlyTeachesOw DepartmentAndCourseNamesAreU
FR-06 Enrolment	UC-11	`AdminService`	ERD	enrolment	S:AdminService.cpp	AdminTest.EnrolOnlyActiveStudents
FR-07 Create assignment	UC-05	`AssignmentService`, `Assignment`	class, ERD	assignment	S:AssignmentService.cpp, U:LecturerMenu.cpp	AssignmentTest.LecturerCreatesAss OnlyTheCoursesLecturerMayCreate ValidatesDeadlineMaxMarkAndTitle
FR-08 Individual extension	UC-04	`AssignmentService`, `Extension`	class, state (Late->Submitted), activity	extension	S:AssignmentService.cpp	AssignmentTest.ExtensionMovesTha RegrantingReplacesTheExtension, E ExtensionReclassifiesUnmarkedLate ExtensionShorterThanLatenessKeep NoExtensionAfterModeration; ST22
FR-09 View assignments	UC-07	`AssignmentService::listForStudent`, `StudentAssignmentView` (v1.1: original deadline, extension kind)	sequence (submit)	assignment, extension, submission	S:AssignmentService.cpp, U:StudentMenu.cpp	AssignmentTest.StudentsSeeOnlyEn ExtensionMovesThatStudentsDeadl
FR-10 Submit	UC-01	`SubmissionService`, `Submission`	sequence (submit), activity, state	submission, submission_version	S:SubmissionService.cpp, U:StudentMenu.cpp	SubmissionTest.MainFlowOnTimeSu ST05-ST07, ST25-ST27
FR-11 Resubmit	UC-01 A1	`SubmissionService`, `SubmissionLifecycle`	sequence alt, state	submission_version	S:SubmissionService.cpp	SubmissionTest.AlternativeFlowRes ExceptionFlowResubmitAfterDeadli LateSubmissionCannotBeResubmitt ExtensionAllowsSubmitAndResubm
FR-12 Late accepted	UC-01 A2	`SubmissionService`	sequence alt, state	submission.status	S:SubmissionService.cpp	SubmissionTest.AlternativeFlowLate LateWorkIsAcceptedEvenWeeksAfte SubmissionExactlyAtDeadlineIsOnTi
FR-13 View released results	UC-06	`SubmissionService::resultsFor`, `StudentResultView`, `SubmissionLifecycle::isVisibleToStudent`	state	submission	S:SubmissionService.cpp, U:StudentMenu.cpp	SubmissionTest.MarkIsHiddenUntilF StudentsNeverSeeEachOthersWork
FR-14 Mark with penalty	UC-02	`MarkingService`, `ILatePenaltyPolicy`, `PercentPerDayPenaltyPolicy`, `LatenessCalculator`	class (1, 2), state, activity	submission	S:MarkingService.cpp, lib/src/policy/LatePenaltyPolicy.cpp, U:LecturerMenu.cpp	UT_LAT (12), UT_PEN (20), IT_MRK/ IT_MRK.PenaltyStrategyIsInjectable
FR-15 Audit log	UC-02, UC-03	`MarkAuditEntry`, `IAuditRepository`	class, ERD, sequences	mark_audit (+ triggers)	S:MarkingService.cpp, S:ModerationService.cpp, sql/001_initial_schema.sql	MarkingTest.OnTimeWorkKeepsRav RemarkRequiresReasonAndIsAudite ModerationTest.AuditTrailRecordsW IT_DB.AuditLogIsAppendOnly
FR-16 Moderate	UC-03	`ModerationService`, `ModerationItem`	sequence (moderate), state, activity	submission, mark_audit	S:ModerationService.cpp, U:HodMenu.cpp	ModerationTest.QueueShowsOnlyN ApproveMovesToModeratedAndAu AdjustChangesMarkWithMandatory
FR-17 Release	UC-03	`ModerationService::releaseAssignment`, `ReleaseSummary`	sequence (moderate), state	submission	S:ModerationService.cpp	ModerationTest.ReleasePublishesO ReleaseCountsUnmarkedWork, Rele
FR-18 View audit	UC-12	`ModerationService::auditTrail/auditLog`	use case	mark_audit	S:ModerationService.cpp, U:HodMenu.cpp, U:AdminMenu.cpp	ModerationTest.AuditLogScope, AdministratorCanInspectAnySubmis

FR	Use case	Classes	Diagrams	Tables	Source files	Te
FR-19 Notifications	UC-15	`NotificationService` (Observer), `SubmissionEventPublisher`	class (2)	notification	S:NotificationService.cpp, lib/src/events/SubmissionEvents.cpp, U:RoleMenu.cpp	UT_EVT (3), NotificationTest (6); ST1
FR-20 Change password	UC-14	`AuthService::changePassword`	-	app_user	S:AuthService.cpp, U:RoleMenu.cpp	AuthTest.ChangePasswordRequires(
FR-22 Class-wide extension *(v1.1, CR1)*	UC-16, UC-07	`AssignmentService::grantClassExtension/deadlineFor`, `ClassExtension`, `ExtensionKind`, `DeadlineInfo`, `NotificationService`	use case, class (1-3), ERD, state, activity, sequence (submit)	class_extension	S:AssignmentService.cpp, S:NotificationService.cpp, lib/src/data/SqliteRepositories.cpp, sql/002_class_extension.sql, U:LecturerMenu.cpp, U:StudentMenu.cpp	ClassExtensionTest (11), IT_DB.UpgradesVersion1DatabaseV
FR-21 First run	UC-08	`AdminService::bootstrapAdministrator`, `Application`	-	app_user	S:AdminService.cpp, U:Application.cpp	AdminTest.BootstrapOnlyAllowedO

Non-functional requirements

NFR	Design element	Verification
NFR-01..03 Performance	indexes in schema; Argon2 parameters; transactions for bulk release	PerformanceTest (5) - `docs/07-testing/reports/performance-results-v1.0.txt`
NFR-04 Password storage	`Argon2idPasswordHasher`, CHECK on `password_hash`	HasherTest (5), UT_SEC.ProductionParametersMeetOwaspMinimum, AuthTest.PasswordsAreStoredAsSaltedArgon2idHashes, IT_DB.SchemaRejectsPlainTextPasswords
NFR-05 Password policy, lockout	`PasswordPolicy`, `AuthService`	UT_SEC/PasswordPolicyCases, AuthTest.Five/*Four*/Unknown*, ST24
NFR-06 Parameterised SQL	`Statement::bind`, repositories	IT_DB.ParameterisedQueriesNeutraliseSqlInjection; code review C3
NFR-07 Audit integrity	triggers, `ITransactionRunner`	IT_DB.AuditLogIsAppendOnly, IT_DB.TransactionRollsBackOnException
NFR-08 Robustness	`LmsError` hierarchy, `RoleMenu::run`, `ConsoleIO` re-prompting	ST04-ST07, ST09, ST10, ST14, ST18, ST19, ST26
NFR-09 Offline, persistence	SQLite file, WAL	IT_DB.DataPersistsAcrossRestarts, AT-01
NFR-10 Usability	numbered menus, manuals	ST17, ST18; AT-06 (pending)
NFR-11 Maintainability	layered design, CI gates	coverage 95.9 %, lizard max CCN 14, cppcheck 0
NFR-12 Portability	CMake, `#ifdef _WIN32` isolation	GCC 16 + Clang 21 builds; CI matrix

Backward check

No test exists without a requirement: unit tests of helpers (`UT_TXT`, `UT_LIF`) trace to NFR-08 (input validation) and the state machine (BR-05/06/07); `IT_DB` traces to NFR-06/07/09.

Appendix B - Test results: test report v1.0, defect log, CR1 regression report

Date: 28 Sep 2026. Build: commit on branch `main` tagged `v1.0`. Raw evidence: `reports/` (GoogleTest text and XML output, performance log, acceptance transcripts, coverage HTML).

1. Summary

Level	Tests	Passed	Failed	Evidence
Unit (UT_*, HasherTest)	145	145	0	`functional-results-v1.0.txt`
Integration (IT_*, service suites)	92	92	0	same
System (ST01-ST27)	27	27	0	same
Functional total	**264**	**264**	**0**	also `functional-results-v1.0.xml`
Performance (NFR-01..03)	5	5	0	`performance-results-v1.0.txt`
Acceptance (AT-01..AT-05)	4 scenarios	4	0	`acceptance/`
Acceptance AT-06 (usability, manual)	1	-	-	not yet executed (needs a volunteer student at the school)

Unit tests include the parameterised cases: 12 lateness boundaries, 11 penalty cases, 52 permission-matrix cases, 25 state transitions and 6 password-policy cases.

Compilers: all functional tests pass with **GCC 16.2** (Debug, `-Werror`, coverage instrumentation) and with **Clang 21** (Zig toolchain, Debug and Release).

2. Coverage of the business-logic layer (`lib/`)

Measured with gcov/gcovr on the GCC build (report: `reports/coverage/index.html`, text: `reports/coverage-summary.txt`).

Metric	Result	Target
Line coverage	**95.9 %** (1 815 / 1 892)	>= 70 %
Function coverage	94.9 % (263 / 277)	-
Branch coverage	59.4 % (1 685 / 2 835)	- (reported for information)

Lowest files: `LmsSystem.cpp` 84 % (`createDefault`, used only by the executable), `DirectoryService.cpp` 90 %, `Database.cpp` 90 % (SQLite bind-failure paths that cannot be triggered with valid statements). Branch coverage is lower because GCC counts the hidden exception-handling branches that the compiler generates for every call that can throw.

3. Performance (Release build, 800 students, production Argon2id)

NFR	Measurement	Limit	Result
NFR-01	Login of student 800	2 000 ms	**153 ms** - PASS
NFR-02	List assignments (20) for a student	1 000 ms	**2.6 ms** - PASS
NFR-02	Submit work	1 000 ms	**1.9 ms** - PASS
NFR-03	HoD moderation queue, 800 items	2 000 ms	**108 ms** - PASS
NFR-03	Release 800 marks (one transaction)	5 000 ms	**338 ms** - PASS

An earlier run on an idle machine measured 56 ms for login. Login time is dominated by Argon2id (by design); the rest is SQLite work, and indexes keep it in milliseconds.

4. Requirement coverage

Every FR-01..FR-21 and NFR-01..NFR-12 has at least one passing verification (see the SRS verification column and `docs/appendices/traceability-matrix.md`). Exceptions: - NFR-10 (usability): the menu structure is verified automatically (ST17, ST18), but the timed first-use test AT-06 is still to be run with a real student. - NFR-12 (portability): verified on Windows with two compilers; Linux and MSVC builds are verified by the CI pipeline once the repository is pushed to GitHub (risk R13).

5. Defects

8 defects found and closed during v1.0 verification (see `defect-log.md`); no open defects at Gate 3.

6. Acceptance observations

- AT-02/03 shows BR-02 working as specified in a way a user might not expect: the demo assignment's deadline was exactly two days before submission, and the work was submitted a few seconds after that point, so it counts as **3 started days late** (30 %). The user manuals explain the rule with examples so students and lecturers are not surprised.

7. Conclusion (Gate 3)

Exit criteria met: all Must requirements implemented and tested, 264/264 functional tests and 5/5 performance tests pass, line coverage of the business-logic layer is 95.9 %, and there are no open Critical/Major defects. **Recommendation: release v1.0.**

Defect Log

Every defect found by testing, analysis or review after the code was first committed. "Found by" names the test or tool, so each entry can be reproduced. All retests were run on the complete functional suite as well as the specific test.

ID	Date	Severity	Found by	Description	Cause	Fix
DEF-01	2026-09-28	Major	Unit test `UT_PEN.RejectsNaNMark`	`PercentPerDayPenaltyPolicy::apply` accepted a NaN raw mark, which would propagate NaN to the final mark.	Range check written as `raw < 0 || raw > max`; every comparison with NaN is false, so NaN passed. `MarkingService` had the correct form, so the defect was reachable only through direct use of the policy.	Commit 28424be: check rewritten as `!(raw >= 0 && raw <=` CS-L7 added to the coding standard. clang-tidy later suggested rev (DeMorgan); rejected and suppressed with a reason.
DEF-02	2026-09-28	Minor (test code)	Performance suite set-up (`SetUpTestSuite`)	Performance fixture could not create 800 students: "CHECK constraint failed: length(username) BETWEEN 3 AND 32".	Generated usernames `s1`…`s9` were shorter than the 3-character minimum. The database constraint correctly rejected them, confirming the schema enforces the rule even when the service layer is bypassed.	Usernames changed to `stu1`…`stu800` (before the first commit of performance test).
DEF-03	2026-09-28	Major (build)	Local build with GCC 16 `-Wshadow -Werror` (the Linux CI configuration)	`test_lateness.cpp` failed to compile: lambda parameter `info` shadows a parameter declared inside the `INSTANTIATE_TEST_SUITE_P` macro. Clang did not warn.	Name clash with a GoogleTest macro internal.	Commit `0e61f89`: parameter renamed `paramInfo`; missing `<cc` added.
DEF-04	2026-09-28	Major (security)	Self code review, finding R1	Login returned faster for an unknown username than for a wrong password (no hash computed), so response time could reveal which usernames exist.	Early return before password verification.	Commit `4b38c14`: verify against a dummy hash when the user is u inactive; regression test `AuthTest.UnknownUserCostsTheSameHashVerificationA`
DEF-05	2026-09-28	Major (security)	Self code review, finding R3	`@file` submission read files of any size into memory and could submit the LMS database itself, exposing password hashes and marks to a lecturer.	No size or path check before reading.	Commit `4b38c14`: regular-file check, database-file check, 100 KB l reading; tests ST25-ST27.
DEF-06	2026-09-28	Minor	Self code review, finding R4	`lms --db` with no file name printed "Unknown argument: --db".	Argument loop required a following value in the same condition.	Commit `4b38c14`: explicit message "--db needs a file name."
DEF-07	2026-09-28	Minor (portability)	Review of MSVC behaviour before enabling Windows CI	`sscanf` (MSVC C4996 deprecation) and `<windows.h>` `min`/`max` macros would break the MSVC build.	Platform-specific headers and functions.	Hand-written date parser; `NOMINMAX` before `<windows.h>`; `-Wer` to the Linux job.
DEF-08	2026-09-28	Minor (maintainability)	lizard, `-C 15`	`time_util::parseLocal` (CCN 19) and `NotificationService::onSubmissionEvent` (CCN 16) exceeded the complexity limit (NFR-11).	Validation and dispatch logic in single functions.	Commit `4b38c14`: extracted helpers. Max CCN is now 14 (`permis` flat switch).

Summary at Gate 3 / v1.0

- 8 defects found, 8 closed; 0 open. By severity: 0 Critical, 5 Major, 3 Minor.
- By detection: unit test 1, fixture/constraint 1, compiler (GCC) 1, code review 3, static analysis/metrics 1, portability review 1.
- Lesson: two security defects (DEF-04, DEF-05) were found by review, not by tests. This supports keeping a review checklist next to automated testing.

CR1 - Implementation and Regression Test Report (release v1.1)

1. Implementation summary

Implemented on branch `feature/cr1-class-extension` exactly as planned in the impact analysis (cr1-change-request.md). No file outside the predicted list was changed.

Area	Change
Schema	`sql/002_class_extension.sql` - new table `class_extension`; applied automatically, `PRAGMA user_version` 1 -> 2
Domain / data	`ClassExtension` struct; `IAssignmentRepository::saveClassExtension/findClassExtension` + SQLite implementation (UPSERT, bound parameters)
Business logic	`AssignmentService::grantClassExtension`, `classExtension`, `deadlineFor` (returns deadline + `ExtensionKind`); `effectiveDeadline` delegates to it; new event `ClassExtensionGranted`; `NotificationService` notifies every enrolled student
UI	Lecturer option 6 "Extend the deadline for the whole class"; extensions list shows the class extension; student list columns *Original deadline*, *Your deadline*, *Extension*
Version	1.1.0
Documents updated	SRS (FR-22, BR-03, glossary), use cases (UC-16, UC-01 step 5), SDD (sections 1, 4.3, 4.4), diagrams (use case, class views 1-3 and overview, ERD, state, activity, submit sequence), traceability matrix, lecturer and student manuals, release notes, risk register, README

No changes were needed in `SubmissionService` or `MarkingService`. Both already asked `AssignmentService` for the effective deadline, so late status, penalties and resubmission picked up class extensions automatically.

2. Regression testing

Run 1 - existing suite unchanged, new code in place

Evidence: `reports/regression-run1-before-test-updates.txt`

Result	Count
Passed	263
Failed	1 - `SystemTest.ST22_LecturerGrantsExtensionShownToStudent`

The failure was **predicted by the impact analysis**. ST22 looked for the v1.0 display text `"<deadline> (extended)"`, which CR1 deliberately replaced with the *Extension* column. The behaviour under test (extension granted, student notified, new deadline shown) was unchanged. The expectation was updated to check the new deadline and the word `individual`. This is an intended specification change, not a regression.

Run 2 - updated and new tests

Evidence: `reports/regression-run2-v1.1.txt`, XML `reports/regression-run2-v1.1.xml`

Suite	Tests	Result
All v1.0 tests (incl. updated ST22 and schema-table list)	264	PASS
New `ClassExtensionTest` (integration)	11	PASS
New `IT_DB.UpgradesVersion1DatabaseWithoutDataLoss`	1	PASS
New system tests ST28, ST29	2	PASS
Total	**278**	**278 PASS, 0 FAIL**

Both compilers: GCC 16.2 (Debug, `-Werror`, coverage) and Clang 21 (Release): 278/278.

New test cases (CR1)

Test	Checks
ClassExtensionTest.AppliesToEveryEnrolledStudent	Every student's deadline and kind = class; original kept
...LaterClassExtensionOverridesEarlierIndividualOne / LaterIndividualExtensionOverridesClassOne	"Latest wins" in both orders; classmates unaffected by individual extensions
...WithoutExtensionsKindIsNone	Default kind and display names
...Validation	Boundary: = original deadline rejected, +1 min accepted; empty reason; other lecturer, student, HoD refused; unknown assignment
...RegrantingReplacesTheClassExtension	Replace semantics; access to the stored reason
...ReclassifiesCoveredLateSubmissions	LATE -> SUBMITTED only when the new deadline covers the submission
...RefusedOnceMarksAreModerated	Allowed while only MARKED, refused after moderation
...LatePenaltyMeasuredFromClassDeadline	No penalty for work inside the class deadline
...StudentsMayResubmitUntilTheClassDeadline	Resubmission window follows the class deadline; closed 1 s after it
...EveryEnrolledStudentIsNotified	Notifications to enrolled students only
IT_DB.UpgradesVersion1DatabaseWithoutDataLoss	Real v1.0 schema + data -> v2; data intact; new table usable

Test	Checks
ST28	Lecturer menu option 6 end to end; both students see *class* and the original deadline
ST29	Student with a later individual extension sees *individual*; classmate sees *class*

Other verification of v1.1

Check	Result	Evidence
Performance NFR-01..03	Login 59 ms, list 1.1 ms, submit 0.5 ms, queue 38 ms, release 120 ms - all PASS	`reports/performance-results-v1.1.txt`
Acceptance AT-01..AT-05 (regression) and new AT-07 (class extension)	5/5 PASS	`reports/acceptance/`
Line coverage of `lib/`	96.1 % (1 896 / 1 973), up from 95.9 %	`reports/coverage-summary-v1.1.txt`
cppcheck / clang-tidy on changed files	0 findings	-
lizard	no function above CCN 15	-

3. Conclusion

CR1 is implemented, every predicted artefact was updated, the complete regression suite passes, and existing v1.0 databases upgrade without data loss. Released as **v1.1**.

Appendix C - User manuals

The administrator prepares the school in the system at the start of each term and helps users with their accounts. The first administrator account is created the first time the program starts (see the installation guide).

```
1. List user accounts
2. Create user account
3. Activate / deactivate an account
4. Reset a user's password (also unlocks)
5. List departments
6. Create department
7. List courses
8. Create course
9. Assign lecturer to course
10. Enrol student on course
11. View mark audit log (whole school)
12. View all my notifications
13. Change my password
0. Log out
```

Setting up a new term (in this order)

1. **Create departments** (6), e.g. "Sciences", "Languages". Names must be unique.
2. **Create user accounts** (2):
3. Choose the role: 1 Administrator, 2 Lecturer, 3 Head of Department, 4 Student.
4. Username: 3-32 characters, letters, digits, `.`, `_`, `-` (e.g. `lmokoena`, `s2026-001`). Must be unique.
5. Lecturers and Heads of Department **must** belong to a department; students may (press Enter for none).
6. Each department can have only **one** active Head of Department.
7. Initial password: at least 8 characters with a letter and a digit. Give it to the user privately and ask them to change it (their menu option "Change my password").
8. **Create courses** (8) with a unique code (e.g. `MAT101`), a title and the department.
9. **Assign lecturers** (9). A lecturer can only teach courses of their own department.
10. **Enrol students** (10) by course ID and student username.

Helping users

- **Forgotten password or locked account:** option 4, "Reset a user's password". This also removes the 15-minute lock.
- **Someone leaves the school:** option 3, answer `n` to deactivate. The account can no longer log in, but all their submissions, marks and audit records are kept. You cannot deactivate your own account.
- **New Head of Department:** deactivate the old HoD first, then create or activate the new one.

Audit log (11)

Shows every mark change in the whole school. Use it when the principal needs to investigate a complaint.

Good practice

- Create one administrator account per person; do not share accounts.
- Make sure the technician takes the daily backup of `lms.db`.
- At the end of the year, deactivate accounts of students who have left and archive their records, following the school's data retention rules (see the professional issues section of the project report).

Lecturer Manual

Log in as described in the Student manual (option `1`, username, password; the password is not shown). You then see new notifications, such as "Neo Mohapi submitted 'Algebra homework 1' (MAT101) LATE.", and the **Lecturer menu**:

```
1. My courses and assignments
2. Create an assignment
3. View submissions for an assignment
4. Mark or re-mark a submission
5. Grant an individual extension
6. Extend the deadline for the whole class      (new in v1.1)
7. View extensions for an assignment
8. View all my notifications
9. Change my password
0. Log out
```

You only see and work with courses the administrator has assigned to you.

2 - Create an assignment

1. Choose the course ID from the list.
2. Type the title and an optional one-line description.
3. Type the deadline as `YYYY-MM-DD HH:MM`, e.g. `2026-10-15 17:00`. It must be in the future.
4. Type the maximum mark (1-1000).

3 - View submissions

Choose the assignment ID. The table shows the submission ID, student, submission time, **Timeliness** ("on time" or "n day(s) late", already taking extensions into account), version, status and mark. Type a submission ID to read the work, or press Enter to skip.

4 - Mark or re-mark a submission

1. Type the **submission ID**.
2. Type the **raw mark**, the mark the work deserves before any late penalty (0 up to the maximum; decimals allowed, e.g. 17.5).
3. Type your **written feedback**. You can use several lines; finish with a line containing only `.`. Feedback is required.
4. **Reason**: leave empty when marking for the first time. When you change a mark you already entered, you must give a reason (e.g. "Missed question 4 on first pass").

The system calculates the late penalty (10 % of the maximum per started day late, zero after 3 days) and shows, for example: `Marked. Raw 20, final 17.5 (1 day late - 10% of 25 = 2.5 deducted).`

The mark is then sent to your Head of Department for moderation. **Students cannot see it until the HoD releases it.** Once the HoD has moderated a mark you can no longer change it; ask the HoD.

Every mark and change is recorded in the audit log with your name, the time and the reason.

5 - Grant an individual extension

1. Choose the assignment ID and type the student's **username**.
2. Type the new deadline (`YYYY-MM-DD HH:MM`); it must be later than the original deadline.
3. Type the reason (e.g. "Medical certificate 12 Oct").

The student is notified and sees "(extended)" next to their deadline. Granting an extension again for the same student replaces the earlier one. If the student had already submitted late but within the new deadline, the work is automatically treated as on time. You cannot grant an extension once the student's mark has been moderated.

6 - Extend the deadline for the whole class (v1.1)

Use this when the whole class needs more time (e.g. a sports day, a power cut, an exam clash). 1. Choose the assignment ID. 2. Type the new deadline (`YYYY-MM-DD HH:MM`), later than the original deadline, and a reason.

Every enrolled student is notified and sees *class* in the Extension column of their assignment list. A student who has an **individual** extension that is even later keeps it. Choosing this option again replaces the class deadline. Work already submitted late but within the new deadline is treated as on time. The option is refused once the HoD has started moderating marks for the assignment.

7 - View extensions

Shows the class extension (if any) and every individual extension for an assignment: student, new deadline, reason, who granted it.

Head of Department Manual

As Head of Department (HoD) you check every mark in your department before students can see it. Log in (option `1`); new notifications such as "A mark for Neo Mohapi on 'Algebra homework 1' (MAT101) awaits moderation." are shown first. Then you see:

```
1. Marks awaiting moderation
2. Approve a mark
3. Adjust a mark
4. Moderated marks ready for release
5. Release moderated marks for an assignment
6. Audit trail of one submission
7. Department mark audit log
8. View all my notifications
9. Change my password
0. Log out
```

1 - Marks awaiting moderation

Lists every marked submission in your department: submission ID, course, assignment (ID and title), student, raw mark, late deduction, final mark / maximum, and the lecturer who marked it.

2 - Approve a mark

Type the submission ID and, optionally, a note (e.g. "Consistent with memo"). The mark becomes *Moderated*. It is **still hidden** from the student.

3 - Adjust a mark

If you disagree with the final mark, type the submission ID, the new final mark and a **reason** (required). The lecturer's raw mark is kept for reference; the audit log records the old and new values. You can adjust a moderated mark again as long as it is not released.

4 and 5 - Release marks

Option 4 shows moderated marks that are ready. Option 5 asks for the assignment ID and a confirmation (`y`). All moderated marks of that assignment are released at once, and each student is notified. The system tells you how many marks are still awaiting moderation or not yet marked; those stay hidden until you moderate and release them later.

Released marks cannot be changed. Check carefully before releasing.

6 and 7 - Audit trail

Shows every change to a mark: time, submission, who changed it, the action (MARK, REMARK, MODERATION_APPROVE, MODERATION_ADJUST, RELEASE), old mark, new mark and reason. Use it to answer questions from students, parents or the principal. The audit log cannot be edited by anyone.

You can only see and moderate work from courses in **your own department**.

Student Manual

Logging in

1. Double-click **School LMS** on the desktop.
2. Type `1` and press Enter to log in.
3. Type your username and your password. Your password is **not shown** while you type; this is normal.
4. If you type the wrong password 5 times, your account is locked for 15 minutes. Ask the administrator if you forgot your password.

After logging in you see any **new notifications** (for example "You have been granted an extension" or "Your mark ... has been released"), then the **Student menu**. Choose an option by typing its number and pressing Enter.

```
1. My courses
2. My assignments and deadlines
3. Submit or resubmit work
4. My results and feedback
5. View all my notifications
6. Change my password
0. Log out
```

Always log out (0) when you finish, so the next person cannot use your account.

2 - My assignments and deadlines

Shows every assignment of your courses, sorted by deadline:

Column	Meaning
ID	Number you type when submitting
Original deadline	The deadline set when the assignment was created
Your deadline	The deadline that applies to you
Extension	- none, `class` the whole class was given more time, `individual` you were given more time personally
Time	Time left, or "deadline passed"
Status	Not submitted, Submitted, Submitted late, Marked (awaiting moderation), Moderated (awaiting release), Released. "(v2)" means you have resubmitted once

3 - Submit or resubmit work

1. Type the assignment **ID**.
2. Type your answer. You can use several lines. When you finish, type a line with only a full stop `.` and press Enter.
3. To submit a file instead, type `@` followed by the file's full path, e.g. `@E:\Maths\homework1.txt`, then a line with `.`. Files may be up to 100 KB.
4. The system tells you the result:
5. **"Submitted on time"** - you may still change your work until the deadline by submitting again.
6. **"Resubmitted (version 2)"** - your new work replaces the old one (the old one is kept on record).
7. **"Submitted LATE"** - your work is accepted but a late penalty will apply.

Late penalty rule

You lose **10 % of the maximum mark for each started day** after your deadline:

Submitted	Days late	Example: maximum 25, you earned 20
before or exactly at the deadline	0	20
1 second to 24 hours late	1	20 - 2.5 = 17.5
24 h + 1 s to 48 hours late	2	15
48 h + 1 s to 72 hours late	3	12.5
more than 72 hours late	4+	0 (you still receive written feedback)

After the deadline you **cannot** change work you already submitted. If you have a good reason for needing more time (illness, family emergency), speak to your lecturer **before** the deadline if possible. An extension moves your deadline and removes the penalty.

4 - My results and feedback

You see your mark and your lecturer's written feedback only **after the Head of Department has checked and released the marks**. Until then the status is "Awaiting marking/moderation". Late deductions are shown next to the mark, e.g. `17.5/25 (raw 20, 1 day(s) late, -2.5)`.

6 - Change my password

Type your current password, then the new one twice. Passwords need at least 8 characters with letters and at least one digit, and must not be the same as your username.

Accessibility

The program uses plain text only, with no colours or pictures, so it works with screen readers (e.g. NVDA or Windows Narrator) and with the Windows magnifier. You can make the text bigger: right-click the window's title bar > *Properties* > *Font*.

Appendix D - Updated risk register

Scales: Likelihood (L) and Impact (I) from 1 (very low) to 5 (very high); **Exposure = L x I** (1-25). Rating: 1-6 Low, 8-12 Medium, 15-25 High. The owner is the person responsible for the mitigation. In this individual project the student owns most risks; stakeholders are named where the mitigation depends on them.

1. Baseline register (Planning, week 1)

ID	Category	Risk (cause -> event -> consequence)	L	I	Exp.	Mitigation / contingency	Owner
R1	Technical	Unfamiliar or missing C++ toolchain / CMake set-up -> build problems -> implementation delayed	3	4	12	Set up CMake, dependencies and CI in the first implementation days; pin dependency versions; document the build	Student
R2	Project	Scope and documentation volume exceed 7 weeks -> late or incomplete deliverables	4	5	**20**	WBS with float; MoSCoW scope rule; write documents alongside each phase, not at the end	Student
R3	Project	Requirements misunderstood or conflicting (e.g. release rule) -> rework after design	3	4	12	Explicit conflict/ambiguity tables; Gate 1 sign-off; traceability matrix	Student / Supervisor
R4	Technical	Weak password storage or SQL injection -> security breach of learner data	2	5	10	Argon2id reference library; prepared statements only; security tests; code review checklist	Student
R5	Technical	Late-penalty/date logic errors (boundaries, time zones) -> unfair marks	3	5	**15**	Precise BR-02 definition; integer epoch arithmetic; EP/BVA unit tests; injectable clock	Student / HoD (rule owner)
R6	Technical	Data loss (disk failure, crash during write) -> assessment records lost	2	5	10	SQLite transactions + WAL; daily backup procedure in installation guide	IT technician
R7	People	Illness or competing coursework of the single developer -> lost days	3	4	12	2 days float; front-load critical-path tasks; keep work in Git daily	Student
R8	Technical / environment	Unreliable internet -> cannot fetch dependencies or push to GitHub	3	3	9	Download dependencies once and reuse (FETCHCONTENT_SOURCE_DIR_*); offline build option; commit locally, push when online	Student
R9	People	Users (teachers) find the system hard to use -> low adoption, errors	3	3	9	Numbered menus, role manuals, demo data for training, usability acceptance test AT-06	Student / Principal
R10	Project	Viva: student cannot explain design decisions -> marks withheld	2	5	10	Keep design rationale in documents; self code review; rehearse the demo script	Student
R11	Technical	Performance on old hardware with 800 students	2	3	6	Indexes; measured NFR tests in CI; tunable Argon2 parameters	Student

2. Update at Gate 1 (requirements)

ID	Change	New L x I	Reason
R3	Reduced	2 x 4 = 8	7 conflicts and 12 ambiguities resolved and recorded; SRS reviewed
R5	Unchanged	3 x 5 = 15	Rule now precise (BR-01/02), but not yet implemented or tested
R12	**New** (Technical): SQLite on a network share corrupts data if the school tries multi-computer use	2 x 5 = 10	Mitigation: single-computer deployment stated in SRS (AS-1, NFR-09) and installation guide; owner: IT technician

3. Update at Gate 2 (design)

ID	Change	New L x I	Reason
R4	Reduced	1 x 5 = 5	Design fixes Argon2id, repository layer with prepared statements, DB CHECK rejecting non-Argon2id hashes
R5	Reduced	2 x 5 = 10	Lateness isolated in `LatenessCalculator` and Strategy classes that are unit-testable with a `FixedClock`
R6	Reduced	2 x 4 = 8	Append-only audit via triggers, transactional services, WAL
R2	Unchanged	4 x 5 = 20	Documentation volume still the main threat

4. Update at Gate 3 (test readiness)

ID	Change	New L x I	Reason
R1	**Occurred, closed**	-	The development PC had no C++ compiler or CMake and installing Visual Studio was not possible. Contingency: portable toolchains (Zig/Clang 21 via pip, later MinGW-w64 GCC 16) and CMake/Ninja via pip. GCC building with -Werror found DEF-03 before CI would have.
R8	**Occurred, closed**	-	Downloads broke repeatedly (pip `IncompleteRead`). Contingency: retries with resume; dependencies pinned by SHA-256 and reused from a local cache through `FETCHCONTENT_SOURCE_DIR_*`.
R5	Reduced	1 x 5 = 5	12 lateness and 11 penalty boundary cases plus 6 integration cases pass; DEF-01 (NaN accepted) found and fixed
R4	Reduced	1 x 5 = 5	Injection tests pass; self-review found and fixed a login timing side channel (R1 in code review)
R11	Closed	1 x 3 = 3	NFR-01..03 measured far inside limits (login 56-153 ms)
R2	Reduced	3 x 4 = 12	Implementation and tests complete; remaining work is documentation and CR1

ID	Change	New L x I	Reason
R13	**New** (Technical): CI on GitHub Actions has never run (no remote configured on the development PC) -> pipeline errors discovered late	3 x 2 = 6	Mitigation: the Linux configuration (GCC, -Werror) was reproduced locally; push early and fix CI before the viva; owner: Student

5. Update after maintenance (v1.1)

ID	Change	New L x I	Reason
R14	**New, closed** (Technical): schema change in CR1 corrupts or loses v1.0 school data during upgrade	2 x 5 -> 1 x 5 = 5	Additive migration 002 only; `IT_DB.UpgradesVersion1DatabaseWithoutDataLoss`; backup step first in the upgrade instructions
R2	Reduced	2 x 4 = 8	All phases delivered; only report/viva remain

6. Current top risks (after v1.1)

1. **R12** network-share misuse (10) - installation guide warning.
2. **R10** viva preparation (10) - demo script (`--seed-demo`, acceptance transcripts), design rationale in the SDD.
3. **R2** schedule/documentation (8) - final report and viva remain.
4. **R13** CI not yet run on GitHub (6) - push the repository and check the Actions run before the viva.

Appendix E - Fully dressed use cases UC-02 and UC-03

UC-02 Mark submission

Field	Content
Primary actor	Lecturer
Stakeholders and interests	Lecturer: record a mark quickly. Student: fair penalty, written feedback. HoD: every mark reaches moderation; changes are traceable.
Trigger	The lecturer chooses "Mark or re-mark a submission".
Preconditions	P1 The lecturer is logged in. P2 The lecturer teaches the course of the submission. P3 The submission is SUBMITTED, LATE or MARKED (not yet moderated).
Postconditions (success)	S1 The submission is MARKED with raw mark, days late, late deduction, final mark, feedback, marker and time. S2 Exactly one audit entry (MARK or REMARK) records who, when, old final mark, new final mark and reason, in the same transaction. S3 The HoD of the department is notified. S4 The mark remains hidden from the student.

Main success scenario 1. The lecturer opens the submissions list of an assignment (ID, student, submission time, on time / days late, version, status, mark) and optionally reads a submission. 2. The lecturer enters the submission ID, a raw mark and multi-line written feedback, leaving the reason blank. 3. The system checks the lecturer teaches the course, the status is markable, 0 <= raw <= maximum and feedback is not empty. 4. The system computes the days late from the submission time and the student's effective deadline *at marking time* (includes *Determine effective deadline*). 5. The system applies the late-penalty policy (includes *Apply late penalty*): 10 % of the maximum per day for days 1-3, 0 after 3 days, never below 0. 6. The system stores the mark and writes the audit entry in one transaction (includes *Record mark audit entry*). 7. The system notifies the HoD and shows "Marked. Raw r, final f (explanation)" and that the mark is hidden until release.

Alternative flows - **A1 Re-mark (extends at step 3, status MARKED):** 2a. The lecturer also enters a reason. 6a. The audit action is REMARK with the previous final mark as old value. - **A2 Late work more than 3 days late:** 5a. Final mark is 0; the raw mark and feedback are still stored and later shown to the student.

Exception flows - **E1 Mark out of range / not a number:** the UI re-prompts; the service rejects NaN or values outside 0..max with "Mark must be between 0 and max". - **E2 Empty feedback:** "Written feedback must not be empty." Nothing is saved. - **E3 Re-mark without reason:** "A reason is required when changing an existing mark." - **E4 Submission already moderated or released:** "This submission has been moderated by the Head of Department and can no longer be changed by the lecturer." - **E5 Lecturer does not teach the course:** "You do not teach the course of this assignment." - **E6 Unknown submission:** "Submission n does not exist."

Business rules: BR-01, BR-02, BR-03, BR-06, BR-09, BR-10, BR-11. **Tests:** MarkingTest.*, IT_MRK.LateMarking (6 boundary cases), ST08-ST10.

UC-03 Moderate and release marks

Field	Content
Primary actor	Head of Department
Stakeholders and interests	Principal/HoD: no mark reaches students unchecked. Lecturer: marks released promptly. Student: fair, final marks with feedback. Regulator: auditable decisions.
Trigger	A notification that marks await moderation, or the HoD chooses "Marks awaiting moderation".
Preconditions	P1 The HoD is logged in. P2 At least one submission of a course in the HoD's department is MARKED.
Postconditions (success)	S1 Each moderated submission is MODERATED with moderator and time; adjusted marks have the new final mark. S2 On release, every MODERATED submission of the assignment is RELEASED and its student notified. S3 Each approval, adjustment and release has an audit entry. S4 MARKED (unmoderated) and unmarked submissions remain hidden.

Main success scenario 1. The system lists MARKED submissions of the HoD's department: submission, course, assignment, student, raw mark, late deduction, final mark / maximum, marker. 2. The HoD selects a submission and approves it, optionally with a note. 3. The system checks department scope and the MARKED status, sets MODERATED and writes an audit entry (MODERATION_APPROVE) in one transaction. 4. Steps 2-3 are repeated until the HoD is satisfied. 5. The HoD lists "Moderated marks ready for release", enters the assignment ID and confirms. 6. The system sets every MODERATED submission of the assignment to RELEASED, writes a RELEASE audit entry for each (one transaction), notifies each student and reports how many were released and how many remain awaiting moderation or unmarked.

Alternative flows - **A1 Adjust mark (extends at step 2):** the HoD enters a new final mark (0..max) and a mandatory reason; the system stores the new final mark, keeps the lecturer's raw mark and writes MODERATION_ADJUST with old and new values. A moderated mark may be adjusted again before release. - **A2 Release cancelled (step 5):** the HoD answers "n"; nothing is released. - **A3 Partial release:** some submissions are still MARKED; they are not released and the summary says so.

Exception flows - **E1 Submission of another department:** "This submission belongs to another department." (BR-08) - **E2 Approve a submission that is not MARKED:** "Only a marked submission that awaits moderation can be approved." - **E3 Adjust without reason / out of range:** validation error; nothing changes. - **E4 Adjust after release:** "Only marked or moderated (unreleased) submissions can be adjusted." (BR-06) - **E5 Unknown assignment or submission:** not-found message.

Business rules: BR-06, BR-07, BR-08, BR-09. **Tests:** ModerationTest.*, NotificationTest.StudentLearnsNothingBeforeRelease, ST11-ST15.
